

WORKING TECHNICAL PROPOSAL

Quantified CUE

A Set-Theoretic Constraint Calculus

Polymorphic functions, dependent specifications,
and representation-independent modules

Aram Hăvărneanu

22 September 2026

Abstract. CUE descriptions are interpreted as predicates on joint environments of possible values. Existential projection explains instance formation and hidden witnesses; universal projection adds obligations over arbitrary values and semantic types. Both are adjoints to substitution, and both preserve refinement. Function types are universal predicates on an operational application relation. Their variance, intersection laws, and polymorphic instances follow from that interpretation. The proposal defines quantified declarations, legal scope extrusion, a conservative data-language embedding, and three language profiles. A worked catalogue covers calling conventions, higher-order programming, dependent invariants, and abstract modules. Existential modules use sealing and equality-respecting simulations to support representation-independent evolution. The reference model establishes refinement, function soundness, and representation-independent replacement under explicit hypotheses. A residual normalizer integrates type obligations with refinement-stable propagation and demanded concrete validation.

Comparison baseline: CUE proposal 4484 and its theory addendum,
repository revision a949162aedb8, inspected 21 September 2026.

Contents

Conventions	iv
1 The existing constraint semantics	1
1.1 Descriptions and completions	1
1.2 The sense in which variability is existential	1
1.3 Universal implementation obligations	2
1.4 The semantic layer of existing features	2
2 Quantification as a set-theoretic operation	3
2.1 Contexts and subjects	3
2.2 Semantic types and universes	3
2.3 Bounds and dependent ranges	4
2.4 Substitution and duality	4
2.5 Empty ranges and uniform subjects	4
3 Unification, floating, and monotonicity	5
3.1 Pointwise unification	5
3.2 Different bounds retain different obligations	5
3.3 Legal scope extrusion	6
3.4 Disjunction and projection	6
3.5 Refinement theorem	6
3.6 Refinable bounds are correlated variables	7
4 Function types from universal obligations	8
4.1 Operational inhabitants and calls	8
4.2 Variance and intersections	8
4.3 Implementations and application refinement	9
4.4 Erased polymorphic functions	9
4.5 Checked calls and effects	9
5 Surface binding and conservative extension	10
5.1 Quantifier expressions	10
5.2 Quantified field declarations	10
5.3 Quantifier blocks	10
5.4 Function-local generics and parametric aliases	11
5.5 A conservative embedding	11

6	Three implementation profiles	13
6.1	Rank, predicativity, and interface boundaries	13
6.2	Full symmetric, higher-rank CUE: $\mathcal{S}_H$	13
6.3	Symmetric, rank-1 CUE: $\mathcal{S}_1$	13
6.4	Universal-only, rank-1 CUE: $\mathcal{U}_1$	14
6.5	Coverage of host-language interfaces	14
6.6	Implementation choice	14
7	Programs and calling conventions	15
7.1	Basic polymorphic constructions	15
7.2	Generic collection programs	16
7.3	Higher-order composition and return values	17
7.4	Quantified records and independent refinement	17
7.5	Argument forms and call packets	18
7.6	Defaults	19
7.7	Open interfaces and partial application	20
7.8	Identity, captured values, and result combination	20
7.9	Validators and iteration	21
8	Dependent descriptions and functions	22
8.1	Dependent contexts	22
8.2	Indexed finite sequences	23
8.3	Matrix dimensions, including empty matrices	24
8.4	Protocol descriptions and refinement predicates	25
8.5	Proof fragments and erasure	26
9	Existentials and modular reuse	27
9.1	A shared hidden representation	27
9.2	Sealing and opening	27
9.3	Two implementations and one client	28
9.4	Unification is an observation	29
9.5	The scope of the evolution guarantee	29
9.6	First-class packages and existential placement	30
9.7	Mixed prefixes and generic modules	31
10	Checking, execution, and formal obligations	32
10.1	Judgments and rigid scopes	32
10.2	Functions and declaration conjunction	32
10.3	Adequate symbolic evaluation	33
10.4	Decision fragments	33

11 Refinement-stable normalization and checking	34
11.1 Hypotheses, refutation, and observation	34
11.2 Stable reductions	34
11.3 Exact residuals and approximate propagation	35
11.4 A finite local kernel	36
11.5 Checking goals are residual constraints	36
11.6 Universal and existential proof search	38
11.7 Counterexamples and remaining environments	38
11.8 Demanded calls and concrete observations	39
11.9 Incremental algorithm	40
11.10 Preservation and persistence	40
11.11 Cost and profile-specific decision fragments	41
12 Feature correspondence and related designs	42
12.1 The comparison baseline	42
12.2 Feature coverage	42
12.3 Three explicit translations	43
12.4 Modular guarantees	44
12.5 CDuce and semantic subtyping	44
12.6 Elixir and checked dynamic execution	44
12.7 Hyperdoctrines and compact closure	44
13 Requirements and implementation outcome	45
13.1 Directionality	45
13.2 Monotone refinement	45
13.3 Cross-file input and output invariants	45
13.4 Callable self-unification	45
13.5 Idiomatic foreign interfaces	45
13.6 Expressive strength and implementation milestones	46
13.7 Conclusion	46
A Core judgments and finite regression model	47
A.1 Formation and binding summary	47
A.2 Sound introduction and elimination principles	47
A.3 Regression checks	48
B Example and feature index	49
References	50

Conventions

Equations and proofs use the semantic reference calculus. Code listings use the proposed surface language and carry profile or extension labels. An expected contradiction is written $_|_$; an unresolved proof or evaluation obligation is written *incomplete*.

Notation	Meaning
V	A fixed set of operational inhabitants, including data and closures.
$\rho \in \text{Env}(\Gamma)$	An assignment to the free variables of context Γ .
$\llbracket T \rrbracket_\rho \subseteq V$	The inhabitants of description T at ρ .
$P \leq Q$	Inclusion of predicates, hence refinement toward more information.
$\top, \perp$	The full subject set and the empty predicate.
$\mathcal{U}_n$	A set of admissible semantic types at universe level n .
$\mathcal{U}_1$	Universal-only, rank-1 surface profile.
$\mathcal{S}_1$	Symmetric rank-1 profile, with scoped existential packages.
$\mathcal{S}_H$	Full symmetric, predicative higher-rank profile.
$\mathcal{D}$	The dependent-value extension of the indicated profile.
$\mathcal{A}$	The opaque-module abstraction discipline.

Universal-only refers to the *new explicit type-binding surface*. Ordinary CUE instance variables retain their existential completion semantics in all three profiles. Rank restrictions concern type quantifiers; dependence on ordinary values is an independent design axis.

The forms `forall (A, B) T` and `exists A { ... }` bind variables in a description. A quantified field `f(A, B): T` constrains one subject at every instance. A parametric alias `S(A) = T` abbreviates a description by capture-avoiding substitution; `S(U)` applies the alias. Instantiation selection `f[U]` refers to the same subject at a universal instance.

The comparison uses proposal 4484 and its addendum at the cited repository revision [5, 6]. Stable CUE expressions retain their existing elaboration and observation rules.

Chapter 1

The existing constraint semantics

1.1 Descriptions and completions

CUE orders descriptions by subsumption and defines unification as their greatest lower bound. At the level of successful data completions, the corresponding set interpretation is inclusion, intersection, and union [1].

Fix a set D of complete data inhabitants. A primitive description denotes a subset of D ; a concrete atom denotes a singleton. Write

$$\llbracket 1 \rrbracket = \{1\}, \quad \llbracket \text{int} \rrbracket = \mathbb{Z}, \quad \llbracket S \ \& \ T \rrbracket = \llbracket S \rrbracket \cap \llbracket T \rrbracket, \quad \llbracket S \mid T \rrbracket = \llbracket S \rrbracket \cup \llbracket T \rrbracket.$$

An open record denotes the records satisfying its field predicates; a closed record additionally constrains its permitted labels. Field absence is a separate possibility from a present field with an unconstrained value.

An open declaration is more accurately a relation than a product of its field marginals. For example:

```
config: {  
  x: int  
  y: x + 1  
}
```

has the relational interpretation

$$R(x, y) \equiv x \in \mathbb{Z} \ \wedge \ y = x + 1.$$

Its two fields cannot be interpreted as independent choices of integers. Projection onto an exported interface hides the other variables existentially. A complete exported record retains its exposed fields as the subject being constrained.

1.2 The sense in which variability is existential

A field declaration $\mathbf{x}: \text{int}$ leaves a witness to be supplied by a completion. Adding $\mathbf{x}: \geq 0$ removes negative witnesses. It imposes no obligation to produce an implementation working uniformly for every integer.

More generally, if Δ names local witnesses and Γ the exposed interface, the description of that interface is

$$\llbracket \exists \Delta P \rrbracket = \{\rho \in \text{Env}(\Gamma) \mid \exists \delta \in \text{Env}(\Delta). P(\rho, \delta)\}.$$

This is the existential reading of ordinary CUE variability. Built-in list and pattern constraints may themselves validate all matching elements; a general binder for an arbitrary semantic type is an additional facility.

Sharing precedes projection. Two contributions to one field elaborate to $\exists x (P(x) \wedge Q(x))$. Two independently introduced witnesses elaborate to $(\exists x P(x)) \wedge (\exists y Q(y))$. These formulas carry different sharing information.

Example 1.1 — Independent instantiations of a description. *Existing CUE*

```
#Cell: {value: int}
left:  #Cell & {value: 1}
right: #Cell & {value: 2}
```

Each instantiation has its own field cells. Within each cell, repeated constraints share that cell. CUE references copy the associated expression and rebind internal references to the corresponding copies [2]. The reference calculus therefore receives an elaborated graph with explicit sharing, rather than equating every textual identifier with one global logic variable.

1.3 Universal implementation obligations

A generic transformation needs one implementation whose behavior is valid for arbitrary types. Write $\text{Map}(A, B, m)$ for the requirement that m applies a supplied transformation from A to B elementwise to a finite A -list and returns the corresponding B -list. The two statements

$$\exists A, B, m. \text{Map}(A, B, m), \quad \exists m. \forall A, B. \text{Map}(A, B, m)$$

are different specifications. The first asks for some usable instance. The second fixes an implementation and requires every instance. In particular, existentially choosing $A = \emptyset$ can trivialize an input obligation. This distinction applies to records as well as functions: a reusable module may contain one operation that works at every element type, and a hidden representation shared by several operations. The language needs to retain both kinds of binding and their dependency order.

1.4 The semantic layer of existing features

The reference model is the *completion layer*. The implementation also retains closure provenance, scope graphs, field-presence information, and preference information. A default-bearing description has successful completions and a preferred subset. Default selection is an observation of that additional structure. Comprehensions elaborate constraints using the host language's established evaluation rules [3].

Legacy programs retain their data observations and evaluation rules. Refinement theorems apply to a fixed elaborated predicate and a fixed quantifier dependency structure.

Chapter 2

Quantification as a set-theoretic operation

2.1 Contexts and subjects

Definition 2.1 (Predicate fiber). *For a context Γ with set of assignments $\text{Env}(\Gamma)$, let $\mathcal{P}_\Gamma = \mathcal{P}(\text{Env}(\Gamma))$, ordered by inclusion. A type expression with subject z denotes a predicate in $\mathcal{P}_{\Gamma, z:V}$.*

The subject is the value described by an expression. A quantifier binds its listed variables and leaves that subject fixed. A record field supplies a projection of the enclosing record as the subject of its field expression.

A substitution $h : \text{Env}(\Delta) \rightarrow \text{Env}(\Gamma)$ induces reindexing $h^*P = h^{-1}(P)$. It preserves all set unions and intersections. For a fixed sort S , let $\pi : \text{Env}(\Gamma) \times S \rightarrow \text{Env}(\Gamma)$ be the projection. Define

$$\exists_\pi P = \{\rho \mid \exists s \in S. (\rho, s) \in P\}, \quad (2.1)$$

$$\forall_\pi P = \{\rho \mid \forall s \in S. (\rho, s) \in P\}. \quad (2.2)$$

These are predicates on the original interface, not families left outside the semantic domain.

Proposition 2.2 (Adjunctions). *For predicates P on the extended context and Q on the base context,*

$$\exists_\pi P \subseteq Q \iff P \subseteq \pi^*Q, \quad \pi^*Q \subseteq P \iff Q \subseteq \forall_\pi P.$$

Proof. The first equivalence expands to: every $(\rho, s) \in P$ has $\rho \in Q$. The second expands to: for every $\rho \in Q$ and every $s \in S$, $(\rho, s) \in P$. $\square$

Thus $\exists_\pi \dashv \pi^* \dashv \forall_\pi$. This is the subset hyperdoctrine underlying the proposal [7, 8].

2.2 Semantic types and universes

Fix a set V of operational values. A universe $\mathcal{U}_n \subseteq \mathcal{P}(V)$ is a set of admissible type interpretations. A valuation assigns $\rho(A) \in \mathcal{U}_n$ to a type variable of sort $\text{Type}(n)$. Type operations are interpreted pointwise in that valuation:

$$\llbracket A \rrbracket_\rho = \rho(A), \quad \llbracket T \wedge U \rrbracket_\rho = \llbracket T \rrbracket_\rho \cap \llbracket U \rrbracket_\rho, \quad \llbracket T \vee U \rrbracket_\rho = \llbracket T \rrbracket_\rho \cup \llbracket U \rrbracket_\rho.$$

A fixed-level admissible family contains the base types and the operations used by the selected fragment. Universes are cumulative and closed at higher levels under the permitted quantified constructions. Taking sufficiently large set-sized families of subsets gives a reference interpretation; an implementation uses finitely represented descriptions of selected members.

For a binder ranging over $\mathcal{U}_n$:

$$\llbracket \forall A T \rrbracket_\rho = \bigcap_{S \in \mathcal{U}_n} \llbracket T \rrbracket_{\rho[A:=S]}, \quad (2.3)$$

$$\llbracket \exists A T \rrbracket_\rho = \bigcup_{S \in \mathcal{U}_n} \llbracket T \rrbracket_{\rho[A:=S]}. \quad (2.4)$$

The quantifier does not replace the subject by a type-indexed family of subjects. These equations are a Curry-style, erased interpretation of quantified *constraints on one value*.

Stratification gives a predicative source discipline. Operational functions will be members of a designated set of closures, rather than the full set-theoretic exponential B^A . This is a Henkin-style choice of realizers and admissible predicates. It lies outside the standard full-function-space extension obstructed by Reynolds's theorem [9]. The equations alone do not assert relational parametricity; that property additionally concerns the operations available to an abstractly typed client [10].

2.3 Bounds and dependent ranges

A type bound $A : T$ abbreviates the range

$$\text{Sub}_n(T, \rho) = \{S \in \mathcal{U}_n \mid S \subseteq \llbracket T \rrbracket_\rho\}.$$

A value binder x in T ranges over $\llbracket T \rrbracket_\rho$. Bounds may depend on earlier variables. Their context extension is the comprehension set

$$\Gamma.D = \{(\rho, d) \mid \rho \in \text{Env}(\Gamma), d \in D_\rho\}.$$

Equations (2.1) and (2.2) then quantify over the fiber D_ρ . This is the same construction for constant and dependent ranges.

For fixed ambient sorts, bounded quantification can be expressed by guards:

$$\exists a \in D P = \exists a (D(a) \wedge P), \quad \forall a \in D P = \forall a (D(a) \Rightarrow P).$$

The implication is a semantic operation in the Boolean predicate model; a surface implementation can retain the guarded clause directly.

2.4 Substitution and duality

For a pullback square of context maps, reindexing commutes with both quantifiers:

$$h^*(\exists_\pi P) = \exists_{\pi'}(h'^*P), \quad h^*(\forall_\pi P) = \forall_{\pi'}(h'^*P).$$

To verify either equation, fix the base assignment and compare the corresponding fibers of the pullback. Their witnesses are in bijection. These Beck–Chevalley laws govern capture-avoiding copying of quantified descriptions.

In each Boolean fiber,

$$\forall_\pi P = \neg \exists_\pi (\neg P).$$

The semantic duality is available even in a surface fragment that exposes only positive conjunctions, disjunctions, and universal binders.

2.5 Empty ranges and uniform subjects

The empty-index conventions are essential:

$$\bigcap \emptyset = V, \quad \bigcup \emptyset = \emptyset.$$

Hence $\forall a \in \emptyset P = \top$ and $\exists a \in \emptyset P = \perp$. If $\mathcal{U}_n$ contains bottom, then $\forall A A = \perp$. If lists are finite and bottom is admissible, $\forall A \text{List}(A)$ consists exactly of the empty list.

Example 2.1 — One subject at every instance. $\mathcal{U}_1$

```
empty(A): [...A]           // []
impossible(A): A           // _|_
impossibleRecord(n in 0 | 1): {
  value: n                 // one field cannot be both 0 and 1
}
```

These are intersections of predicates on one list, value, or record. A parametric alias abbreviates a description by substitution. A descriptor-producing function instead returns a description as a value.

Chapter 3

Unification, floating, and monotonicity

3.1 Pointwise unification

For a common range D ,

$$(\forall a \in D P(a)) \wedge (\forall a \in D Q(a)) = \forall a \in D (P(a) \wedge Q(a)). \quad (3.1)$$

This follows by regrouping intersections. Binder names are alpha-equivalent.

Example 3.1 — Refining a polymorphic record across declarations. $\mathcal{U}_1$

```
r(A): {wrap: func(A) -> {value: A}}
r(B): {wrap: func(B) -> {tag: "wrapped"}}

// Unified description, after alpha-renaming:
r(T): {wrap: func(T) -> {value: T, tag: "wrapped"}}
```

The two output obligations constrain one operation at each T .

Equal spelling does not identify independent existential witnesses:

$$(\exists a P(a)) \wedge (\exists b Q(b)) = \exists a, b (P(a) \wedge Q(b)).$$

Shared existential witnesses are introduced by a shared scope and are conjoined before their binder is discharged.

3.2 Different bounds retain different obligations

For a common body P ,

$$(\forall a \in D P(a)) \wedge (\forall a \in E P(a)) = \forall a \in D \cup E P(a).$$

Here $D \cup E$ is a union of *assignments*. For type bounds,

$$\text{Sub}_n(B) \cup \text{Sub}_n(C) \subseteq \text{Sub}_n(B \cup C)$$

can be strict: mixed subtypes of $B \cup C$ need lie below neither arm.

Example 3.2 — Two bounds do not become one upper bound. $\mathcal{U}_1$

```
choose(A: int): func(A, A) -> A
choose(A: string): func(A, A) -> A

// The stronger declaration also covers mixed unions:
stronger(A: int | string): func(A, A) -> A
```

The first pair retains both guards. With different bodies, its meaning is $\forall A ((A \subseteq \mathbb{Z} \Rightarrow P(A)) \wedge (A \subseteq \text{String} \Rightarrow Q(A)))$.

3.3 Legal scope extrusion

For $a \notin \text{FV}(Q)$, existential Frobenius reciprocity gives

$$(\exists a P) \wedge Q = \exists a (P \wedge Q).$$

For a nonempty fixed range, the universal counterpart holds:

$$(\forall a P) \wedge Q = \forall a (P \wedge Q).$$

For a possibly empty dependent range, the universally quantified expression must retain Q outside the guard. Implementations store the guard and the independent conjunct separately until inhabitation is known.

Example 3.3 — An ordinary field floats as a constant obligation. $\mathcal{U}_1$

```
r(A): {id: func(A) -> A}
r:    {label: "utilities"}

// Type universes are nonempty:
r(A): {id: func(A) -> A, label: "utilities"}
```

Independent same-kind quantifiers commute. An existential and a universal commute only when a separately justified independence law applies. In general,

$$\exists y \forall x (y = x) \neq \forall x \exists y (y = x).$$

The dependency graph records which universal choices a witness may depend on. File order is irrelevant; dependency order is semantic.

3.4 Disjunction and projection

Existential projection preserves unions; universal projection preserves intersections. Their opposite distributivities fail.

Example 3.4 — Binder scope across a union. $\mathcal{U}_1 + \mathcal{D}$

```
a: forall (n in 0 | 1) (n | (1 - n)) // 0 | 1
b: (forall (n in 0 | 1) n) |
   (forall (n in 0 | 1) (1 - n))    // _|_
```

Projection of an entire record must also retain joint witnesses. Take $R_0 = \{(s, 0)\}$ and $R_1 = \{(s, 1)\}$. Projection p onto the first coordinate satisfies

$$p(R_0 \cap R_1) = \emptyset, \quad p(R_0) \cap p(R_1) = \{s\}.$$

Pushing record selection through a universal intersection is therefore only an inclusion in general. Pointwise unification operates on the whole subject, including its correlated fields.

3.5 Refinement theorem

Theorem 3.1 (Monotone quantified refinement). *Let $P' \subseteq P$ be predicates on one elaborated context, with the same quantifier dependency graph. Every legal composite Q of existential and universal projections satisfies $Q(P') \subseteq Q(P)$. In particular, $Q(P \wedge R) \subseteq Q(P)$.*

Proof. Both quantifiers are monotone, either by definition 2.2 or directly from their membership clauses. Composition preserves monotonicity. $\square$

A contradictory closed predicate remains contradictory. A projected atomic result constrained to $\{v\}$ can remain $\{v\}$ or become empty. Its successful completion cannot change to a different atom.

3.6 Refinable bounds are correlated variables

Let an existential semantic type X have upper bound `Number`, and let

$$P(X, f) \equiv X \subseteq \text{Number} \wedge \forall A \subseteq X T(A, f).$$

Adding $X \subseteq \mathbb{Z}$ gives $P' = P \wedge (X \subseteq \mathbb{Z}) \subseteq P$. The result remains monotone after existentially hiding X . The universal obligation for one fixed X is contravariant in its bound; the *joint predicate on possible X and f* is refined by conjunction. A solver retains X and its correlations. It does not substitute its current upper approximation into every occurrence.

Example 3.5 — A particular client and a universally capable producer. $\mathcal{U}_1$

```
job: {
  x: number
  f: func(x) -> string
  result: f(x)
}
job: {x: int & >=0}
job: {x: 7, f: func(n: int) -> string: "\n(n)"}
```

In a complete instance, `x` denotes one integer and the function obligation uses its singleton type. The joint relation retains that dependency. A separate `func(number) -> string` requires all numeric inputs. **Implementation obligation.** The elaborator distinguishes an ordinary witness reference, whose type-position use is a singleton, from a type-sorted variable, whose use is its decoded semantic type. Copying a CUE description preserves that classification and the copied witness graph.

Chapter 4

Function types from universal obligations

4.1 Operational inhabitants and calls

A closure is an operational value with code, a captured environment, and a fixed argument-binding protocol. Let $F \subseteq V$ be the set of such values. A *call packet* records positional arguments, labeled arguments, and explicit omission. A deterministic partial evaluator has observations

$$\text{run}(f, p) \in \{\text{Ok}(S), \text{Reject}, \text{Raise}(e)\},$$

where $S \subseteq V$ is a set of successful result completions. A concrete result v gives $S = \{v\}$. Evaluation can also remain unresolved; unresolved evaluation is a state of computation, rather than a proof that its completion set is empty. The terminating reference fragment uses structurally bounded evaluation.

Define the successful-call predicate

$$G_B(f, p) \equiv \exists S. \text{run}(f, p) = \text{Ok}(S) \wedge \emptyset \neq S \subseteq B.$$

A function contract is then

$$A \Rightarrow B = \{f \in F \mid \forall p \in A. G_B(f, p)\}. \quad (4.1)$$

Here A is a set of complete call packets, derived from the parameter surface. Single positional arguments can be identified with their one-element packets. The surface $\text{func}(A) \rightarrow B$ denotes this set.

A body supplies one operational inhabitant. Contract membership asserts its coverage and result property. The predicate does not perform program synthesis. For an empty domain, $\emptyset \Rightarrow B = F$. For nonempty A , $A \Rightarrow \emptyset = \emptyset$ in the successful-call interpretation.

4.2 Variance and intersections

Proposition 4.1 (Arrow laws). *For all domain and result sets:*

$$\begin{aligned} A \subseteq C, D \subseteq B &\implies (C \Rightarrow D) \subseteq (A \Rightarrow B), \\ (A \cup C) \Rightarrow B &= (A \Rightarrow B) \cap (C \Rightarrow B), \\ A \Rightarrow (B \cap D) &= (A \Rightarrow B) \cap (A \Rightarrow D). \end{aligned}$$

Proof. The first two statements restrict or split the universal input obligation. For the third, determinism fixes the same result description S at each packet; requiring $S \subseteq B$ and $S \subseteq D$ is equivalent to requiring $S \subseteq B \cap D$, with the same success and nonemptiness conditions. $\square$

Thus $(\mathbb{Z} \Rightarrow \text{String}) \cap (\text{Number} \Rightarrow \text{String}) = \text{Number} \Rightarrow \text{String}$. More generally, an intersection of arrows preserves input/output correlations that a single arrow may lose.

Example 4.1 — A correlation-preserving overloaded contract. $\mathcal{U}_1$

```
convert: (func(int) -> string) & (func(string) -> int)
```

One implementation must meet both obligations. A body may perform explicit case analysis. The type intersection itself performs no branch selection. On overlapping domains, all result obligations apply.

4.3 Implementations and application refinement

For a complete closure f , unification with T denotes $\{f\} \cap \llbracket T \rrbracket$. A successful conformance check preserves the closure. An incompatibility makes that intersection empty. A symbolic closure carries unresolved environment and conformance predicates until sufficient information is available.

For a set of closures X and input packets Q , define

$$\text{App}(X, Q) = \bigcup \{S \mid f \in X, p \in Q, \text{run}(f, p) = \text{Ok}(S)\}.$$

Joint environments are retained when X and Q share variables. Then

$$\text{App}(X \cap Y, Q) \subseteq \text{App}(X, Q) \cap \text{App}(Y, Q). \quad (4.2)$$

Membership on the left has one closure witness that works on both sides. Equality can fail: distinct closures can agree at a particular input while having empty singleton intersection.

Example 4.2 — Capabilities and one-call constraints. $\mathcal{U}_1$

```
wide: func(number) -> string
wide: func(x: int) -> string: "ok" // incompatible coverage

narrow: func(int) -> string
narrow: func(x: number) -> string: "ok" // conforming implementation
```

The second contract states a minimum capability. A particular call domain can be constrained with an existential argument cell as in chapter 3. This is the quantifier distinction between clients and producers.

4.4 Erased polymorphic functions

A polymorphic identity has the set interpretation

$$I = \bigcap_{A \in \mathcal{U}_n} (A \Rightarrow A).$$

If all relevant singletons are admissible, then $f \in I$ implies $\text{run}(f, x) = \text{Ok}(\{x\})$ for each x : instantiate at $\{x\}$. If the admissible family is coarser, the statement only forces preservation of its available predicates.

Example 4.3 — A scheme and an ordinary arrow unify normally. $\mathcal{U}_1$

```
id(A): func(A) -> A
id: func(int) -> int // redundant instance

bad(A): func(A) -> A
bad: func(int) -> bool // _|_: int and bool are disjoint
```

Universals distribute through these conjunctions by equation (3.1). An ordinary instantiation is a consequence of the universal constraint, rather than its replacement.

4.5 Checked calls and effects

For a set ϵ of admitted error tags, define

$$A \Rightarrow_\epsilon B = \{f \in F \mid \forall p \in A. G_B(f, p) \vee \exists e \in \epsilon. \text{run}(f, p) = \text{Raise}(e)\}.$$

The pure arrow has $\epsilon = \emptyset$. Domain rejection is a separate observation from a permitted failure within the domain. A bridge supporting logical integers with representability failures has type $\mathbb{Z} \Rightarrow_{\text{bridge}} \mathbb{Z}$. It does not thereby support arbitrary fractional inputs. Divergence, when admitted by an execution profile, is a separately named outcome.

Chapter 5

Surface binding and conservative extension

5.1 Quantifier expressions

The core surface expressions are

```
forall (A, B) T
exists A { ... }
forall (A: number) func(A, A) -> [A, A]
forall (n in int & >=0) T
```

Parentheses group multiple binders or a constrained binder; a single simple binder may omit them. The body is the following expression. A record body is already delimited by braces. Quantifiers have low expression precedence, and parentheses delimit narrower scopes inside unions and arrows.

Unadorned A is type-sorted, at an inferred universe. $A: T$ is an upper bound on that semantic type. $x \text{ in } T$ binds an ordinary value. An explicit $A \text{ in } \text{Type}(n): T$ specifies both the universe and the upper bound. All binders are lexical and alpha-renamable. Bounds may reference earlier binders and outer variables.

5.2 Quantified field declarations

The declaration

```
map(A, B): func(func(A) -> B, [...A]) -> [...B]
```

elaborates exactly to

```
map: forall (A, B) func(func(A) -> B, [...A]) -> [...B]
```

The field value is fixed outside the quantifier. The outer record's field selection is retained as the subject of the quantified predicate.

For a quantified record:

```
r(A): {
  id: func(A) -> A
  wrap: func(A) -> {value: A}
}
```

the order is $\exists r \forall A P(r, A)$ when the surrounding record is closed existentially. Both projections refer to the same record for all A . The field-sugar form is concise for rank-1 declarations; RHS syntax also permits nested and higher-rank positions.

5.3 Quantifier blocks

A lexical binder prefix can be written in a record block:

```
x: {
  exists T: U
  forall V: W
  // The whole following declaration body is in this prefix.
  value: T
  transform: func(V) -> V
}
```

This elaborates to

```
x: exists (T: U) forall (V: W) {
  value: T
  transform: func(V) -> V
}
```

The ordered prefix records dependency. The declarations after the prefix are conjoined without source-order dependence. Contributions from other files unify with the entire quantified subject; they do not capture these private binder names merely by spelling them alike. A named interface may explicitly export a sharing equation when such sharing is intended.

A syntactically asymmetric alternative requires external parentheses for universals and block prefixes for existentials:

```
x(V: W): {
  exists T: U
  value: T
  transform: func(V) -> V
}
```

Its prefix is $\forall V \exists T$, rather than $\exists T \forall V$. The two spellings express different dependence. Both quantifiers still use the dual semantic operations of chapter 2.

5.4 Function-local generics and parametric aliases

A compact function-only spelling is possible:

```
map: func<A, B>(func(A) -> B, [...A]) -> [...B]
```

It expands to an outer universal on the function subject.

A parametric alias binds parameters in a description:

```
Box(A) = {value: A}
Pair(A, B) = [A, B]
```

The = form defines an alias; the : form constrains a field. Alias application substitutes its arguments capture-avoidingly into the body, so `Box(int)` expands to `{value: int}`. Alias parameters use the same sort and bound syntax as quantified binders; arguments must satisfy those bounds. A nonparametric alias uses the existing form `let S = T`.

In contrast, `box(A): {value: A}` requires one value lying in every A and is bottom when bottom is admissible. Reified descriptor functions retain their descriptor arguments as correlated variables.

`foo[T]` selects the instance of a universally constrained subject justified by substituting T for its binder. For a universal description, selection refers to the same inhabitant at that instance; it retains the subject and its universal constraints. Thus `id[int](3)` uses the integer instance of `id`. Selection is erased. Implicit instantiation uses fresh flexible variables at the call site and retains the universal declaration.

5.5 A conservative embedding

Definition 5.1 (Legacy-data observation). *A legacy-data observation is evaluation and export using the pre-extension operators and data formats, applied to a program in the legacy syntax. Its completion domain is D , embedded into the extended V by $i : D \hookrightarrow V$.*

Theorem 5.2 (Relative conservativity). *Suppose the extension retains the legacy elaborator, base evaluation rules, default-selection rules, and data constructors. For every legacy description T and legacy assignment ρ ,*

$$i^{-1}(\llbracket T \rrbracket_{i\rho}^+) = \llbracket T \rrbracket_{\rho}^0.$$

Closed legacy programs have the same legacy-data observations under the extended evaluator.

Proof. For data predicates, the claim is the defining restriction of each extended primitive to D . Inverse image preserves intersections and unions. Record and list constructors preserve the embedding and the field-sharing elaboration. Existing local projection and copying operations are unchanged. The operational statement follows by induction on the unchanged evaluation derivation, with identical default selection and export. $\square$

The theorem is observational and data-relative. Extended top additionally admits new kinds of inhabitants in new-language contexts. This distinguishes conservativity from equality of global carriers.

Design rule. Gate new binder and arrow forms by language version or experiment attributes. Treat `forall`, `exists`, and `func` as contextual keywords in the selected syntax. Existing data files retain their parsing, closing, copying, and evaluation behavior.

Existing struct-as-function code remains a relational program:

```
#Add: {a: int, b: int, out: a + b}
sum: (#Add & {a: 1, b: 2}).out
```

A function declaration adds a reusable universal obligation and a concise call form. Refactoring to that declaration is an explicit program change, checked against the resulting contract.

Chapter 6

Three implementation profiles

6.1 Rank, predicativity, and interface boundaries

Type rank measures the placement of polymorphic binders relative to function arguments. A callback required to work at all types is higher-rank; an ordinary higher-order function such as `map` can have a rank-1 type. Predicativity is a separate restriction on the universes admitted as instantiations.

For the restricted profiles, use a *boundary-prenex* discipline. Within one executable signature clause, type quantifiers occur in an outer prefix over a quantifier-free body, after justified normalization. Module interfaces are explicit additional boundaries. Finite intersections retain several prefixed clauses when their bounds or dependencies differ. An alias expands for the rank check; naming a higher-rank type does not change its rank.

A positive universal may sometimes float through a result arrow when its variable is absent from the input and the established deterministic arrow law justifies the move. Quantifiers over argument types retain their position. Existentials whose witnesses depend on arguments likewise retain that dependence.

6.2 Full symmetric, higher-rank CUE: $\mathcal{S}_H$

$\mathcal{S}_H$ admits both quantifiers at arbitrary well-sorted type positions, with the predicative universe discipline. It includes first-class existential packages, polymorphic arguments, and nested quantified records.

```
use: func(forall A func(A) -> A) -> [int, string]

#Showable: exists A {
  value: A
  show: func(A) -> string
}
show: func(#Showable) -> string
```

The checker uses explicit introduction and elimination rules, scope levels, and annotations. Higher-rank bidirectional checking provides a useful proof-theoretic organization [20], while Boolean connectives and dependent predicates require their own sound constraint procedures.

6.3 Symmetric, rank-1 CUE: $\mathcal{S}_1$

$\mathcal{S}_1$ admits existential and universal prefixes at declaration and module-interface boundaries. An executable callback parameter is monomorphic at each call. Abstract modules are opened at their module boundary, exposing a rigid local representation name and monomorphic operations to their clients. Passing an inline quantified package to a function, or receiving a universally quantified callback, uses $\mathcal{S}_H$.

A representative interface is

```
Counter: exists State {
  zero: State
  next: func(State) -> State
  read: func(State) -> int
}
```

Module-level client code opens `Counter`, then passes `next` and `read` as ordinary monomorphic functions. The prefix can contain both kinds of binder, with explicit dependency order. Symmetry

concerns their availability and semantic laws at the same boundaries; it does not authorize commuting them.

This profile supports ML-style abstract module reuse with a boundary-prenex rank restriction.

6.4 Universal-only, rank-1 CUE: $\mathcal{U}_1$

$\mathcal{U}_1$ adds only explicit outer universal type binders. Existing instance variables, local field cells, and call-instantiation variables remain existentially interpreted. The surface need not expose existential type packaging to express identity, mapping, composition, filtering, folds supplied as primitives, and many generic adapter interfaces.

The same semantics still defines existential projection internally. The implementation simply exposes a smaller formation and elimination interface. It therefore gains universal capability obligations and compositional polymorphic subtyping without requiring an opaque-module boundary in its first release.

```
id(A): func(A) -> A
compose(A, B, C): func(func(B) -> C, func(A) -> B) ->
    func(A) -> C
map(A, B): func(func(A) -> B, [...A]) -> [...B]
```

These declarations require rank-1 universals. They already distinguish reusable implementations from existentially constrained calls.

6.5 Coverage of host-language interfaces

Rank-1 signatures describe ordinary generic APIs. Higher-rank callbacks, abstract packages, and some lifetime-polymorphic or dependent APIs require a richer representation. GHC’s documented distinction between rank-1 schemes and polymorphic parameters is a direct example [23]. A host binding must either express its actual quantifier structure or export a deliberately weaker interface with that loss recorded.

Facility	$\mathcal{U}_1$	$\mathcal{S}_1$	$\mathcal{S}_H$
Outer polymorphic arrows and records	yes	yes	yes
Finite intersections of scheme clauses	yes	yes	yes
Existential instance variables	yes	yes	yes
Explicit abstract module type witnesses	—	yes	yes
Polymorphic callback parameters	—	—	yes
First-class nested quantified packages	—	—	yes
Value dependence as a separate extension	$\mathcal{D}$	$\mathcal{D}$	$\mathcal{D}$

6.6 Implementation choice

$\mathcal{U}_1$ supports rank-1 generic interfaces. $\mathcal{S}_1$ adds explicit modular abstraction at stable boundaries. $\mathcal{S}_H$ admits quantified descriptions at arbitrary well-sorted positions. The inclusion of source fragments is conservative on programs already admitted by the smaller profile. A rank restriction does not by itself decide arbitrary semantic subtyping, quantified arithmetic, or program equivalence; each checker specifies its supported predicate fragment.

Chapter 7

Programs and calling conventions

7.1 Basic polymorphic constructions

Example 7.1 — Identity at unrelated instances. $\mathcal{U}_1$

```
id(A): func(x: A) -> A: x
integer: id(7)           // 7
text:    id("seven")     // "seven"
record:  id({port: 443})  // {port: 443}
chosen:  id[int & >=0](7) // a checked explicit instance
```

One closure meets every instance. Explicit instantiation does not construct a wrapper or remove other valid instances.

Example 7.2 — Pairing preserves an inferred common refinement. $\mathcal{U}_1$

```
pair(A: number): func(x: A, y: A) -> [A, A]: [x, y]
p: pair[int & >=1](2, 5)           // [2, 5]
q: pair(1, 2.5)                   // [1, 2.5]
```

The mixed call has the admissible instance $A = \{1, 2.5\}$. A shared type variable preserves a common refinement; it does not enforce equal primitive tags. A constant result $[0, 0]$ would satisfy the unrefined numeric result type but would fail the instance $A = \{2, 5\}$. Matching primitive categories can instead be specified by the input relation $[\mathbb{Z}, \mathbb{Z}] \cup [\text{Float}, \text{Float}]$.

Example 7.3 — Selecting a value preserves its subtype. $\mathcal{U}_1$

```
min(A: number): func(x: A, y: A) -> A: {
  if x <= y {out: x}
  if x > y  {out: y}
}.out
m: min[int & >=10](12, 20)    // 12
```

Either branch returns an input inhabitant. By contrast, the proposed contract `forall1 (A: number) func(A) -> A` does not justify adding one: $A = \{0\}$ is a counterexample. Numeric bounds permit numeric operations; result refinements require their own proof.

Example 7.4 — Two type parameters preserve asymmetric information. $\mathcal{U}_1$

```
swap(A, B): func(p: [A, B]) -> [B, A]: [p[1], p[0]]
s: swap([7, "seven"])         // ["seven", 7]
```

The two coordinates retain their distinct type variables, rather than being merged into one union.

7.2 Generic collection programs

Example 7.5 — Map with a single implementation. $\mathcal{U}_1$

```
map(A, B): func(g: func(A) -> B, xs: [...A]) -> [...B]: [
  for x in xs {g(x)}
]
inc: func(x: int) -> int: x + 1
text: func(x: int) -> string: "\x"

numbers: map(inc, [1, 2, 3]) // [2, 3, 4]
strings: map(text, [1, 2, 3]) // ["1", "2", "3"]
```

The body is checked with rigid A, B . Each call creates flexible instance variables. Finite sets of admissible instantiations can contribute intersected typings, as in the CDuce polymorphic application discipline [15].

Example 7.6 — Filtering and partitioning preserve elements. $\mathcal{U}_1$

```
filter(A): func(p: func(A) -> bool, xs: [...A]) -> [...A]: [
  for x in xs if p(x) {x}
]
partition(A): func(p: func(A) -> bool, xs: [...A]) -> {
  yes: [...A]
  no: [...A]
}: {
  yes: [for x in xs if p(x) {x}]
  no: [for x in xs if !p(x) {x}]
}
positive: func(x: int) -> bool: x > 0
kept: filter(positive, [-2, 0, 4]) // [4]
```

The callback is pure. An effectful callback requires an explicit effect policy, including the meaning of repeated evaluation.

Example 7.7 — Flattening a generic transformation. $\mathcal{U}_1$

```
flatMap(A, B): func(g: func(A) -> [...B], xs: [...A]) -> [...B]: [
  for x in xs for y in g(x) {y}
]
duplicate(A): func(x: A) -> [A, A]: [x, x]
twice: flatMap(duplicate, [1, 2]) // [1, 1, 2, 2]
```

Example 7.8 — Mapping named record coordinates. $\mathcal{U}_1$

```
mapPoint(A, B): func(g: func(A) -> B, p: {x: A, y: A}) -> {
  x: B
  y: B
}: {
  x: g(p.x)
  y: g(p.y)
}
p: mapPoint(text, {x: 2, y: 3}) // {x: "2", y: "3"}
```

Open/closed record constraints retain their existing meaning. A separate row parameter can preserve additional fields when a row-polymorphic fragment is selected.

7.3 Higher-order composition and return values

Example 7.9 — Composition is rank-1. $\mathcal{U}_1$

```
compose(A, B, C): func(g: func(B) -> C, f: func(A) -> B) ->
  func(A) -> C:
  func(x: A) -> C: g(f(x))

pipeline: compose(text, inc)
answer: pipeline(4)           // "5"
```

The returned closure captures f and g . Its generic variables are rigid assumptions while checking the body; their run-time representations are erased.

Example 7.10 — A value-dependent closure. $\mathcal{U}_1$

```
withPrefix: func(prefix: string) -> func(string) -> string:
  func(s: string) -> string: prefix + s

warn: withPrefix("warning: ")
message: warn("retry")           // "warning: retry"
```

Two closures from the same body with different concrete prefixes have different captured environments.

Example 7.11 — A polymorphic callback. $\mathcal{S}_H$

```
useBoth: func(p: forall A func(A) -> A) -> [int, string]: [
  p(7),
  p("seven"),
]
ok: useBoth(id)           // [7, "seven"]
onlyInt: func(x: int) -> int: x
bad: useBoth(onlyInt)     // incompatible universal obligation
```

The callback itself is polymorphic. A new instantiation of the surrounding function for each use of p would not prove this contract.

Example 7.12 — A nested universal result. $\mathcal{S}_H$ syntax; *prenex normalization may apply*

```
constant(A): func(x: A) -> (forall B func(B) -> A):
  func(_) -> A: x

seven: constant(7)
a: seven("ignored")           // 7
b: seven(true)                // 7
```

The resulting closure depends on x and works uniformly for every subsequent argument type. Moving B outward is permitted only through the proven positive arrow law, with B absent from the domain.

7.4 Quantified records and independent refinement

Example 7.13 — A reusable polymorphic utility record. $\mathcal{U}_1$

```
utilities(A): {
  id: func(x: A) -> A: x
  wrap: func(x: A) -> {value: A}: {value: x}
}
utilities: {name: "core"}

wrapped: utilities.wrap("payload") // {value: "payload"}
```

The name field is a constant conjunct on the same record.

Example 7.14 — A cross-file behavioral strengthening. $\mathcal{U}_1$

```
// interface.cue
wrap(A): func(A) -> {value: A}

// metadata.cue
wrap(A): func(A) -> {tag: "wrapped"}

// implementation.cue
wrap(A): func(x: A) -> {value: A, tag: "wrapped"}: {
  value: x
  tag: "wrapped"
}
```

The body is checked against both pointwise result constraints. No declaration replaces or takes precedence over another.

Example 7.15 — An intersection of domain-specific guarantees. $\mathcal{U}_1$

```
classify: func(x: int) -> ("negative" | "nonnegative"): {
  if x < 0 {out: "negative"}
  if x >= 0 {out: "nonnegative"}
}.out
classify: (func(int & <0) -> "negative") &
  (func(int & >=0) -> "nonnegative")

c: classify(-3)           // "negative"
```

An ordinary union of function values describes alternatives, whereas this intersection describes one value satisfying both implications.

Example 7.16 — Unresolved alternatives and defaults. $\mathcal{U}_1$

```
f: (func(x: int) -> int: x) |
  (func(x: int) -> int: x + 1)
a: f(3)           // incomplete operational choice

preferred: *(func(x: int) -> int: x) |
  (func(x: int) -> int: x + 1)
b: preferred(3)   // 3, under legacy default selection
```

Constraint propagation can retain the set of alternative completions. Execution uses the ordinary concreteness/default discipline to select a callable value.

7.5 Argument forms and call packets

The five parameter forms in proposal 4484 all elaborate to packet predicates [5]. Named parameters admit a positional packet or the matching label; required and optional name-only parameters use labels; anonymous and aliased positional parameters use positions only. Packet binding precedes value constraint solving.

Example 7.17 — Positional, named, and mixed calls. $\mathcal{U}_1$

```
sum: func(a: int, b: int) -> int: a + b
p: sum(2, 3)           // 5
q: sum(a: 2, b: 3)     // 5
r: sum(2, b: 3)        // 5
```

Example 7.18 — Name-only required and optional parameters. $\mathcal{U}_1$

```
key: func(a!: int, note?: string) -> int: a
x: key(a: 4)                      // 4; note is absent
z: key(a: 4, note: "audit") // 4
bad: key(4)                      // invalid packet: a is name-only
```

A body that reads an absent optional parameter must prove presence or produce a checked absence failure. Optionality is an absence alternative, not an unknown present value.

Example 7.19 — Positional-only aliases and anonymous parameters. $\mathcal{U}_1$

```
scale: func(_~value: int, _: int) -> int: value * 2
x: scale(5, 99)                  // 10
bad: scale(value: 5, 99)         // invalid label/order
```

The positional alias names a local body variable without adding a public label. Renaming it preserves the packet protocol.

Example 7.20 — Hidden labels and attributes. $\mathcal{U}_1$

```
package internal
measure: func(_value: int @go(Int), unit!: string) -> int: _value
inside: measure(_value: 7, unit: "ms")
// Another package can use the positional slot, not the hidden label.
```

Hidden labels include their package identity. Attributes are retained as metadata; only a declared extension assigns them operational significance.

Example 7.21 — Errors distinguish presence from values. $\mathcal{U}_1$

```
a: sum(1, a: 1)                  // duplicate binding, even with equal values
b: sum(1, 2, 3)                  // surplus positional argument
c: sum(a: 1, b: 2, c: 3)         // unknown label
d: sum(1)                        // required argument missing
e: sum(_, 2)                     // supplied but incomplete, not missing
```

A duplicate packet is invalid before field unification. Conjoining equal values does not erase a duplicate-binding error.

7.6 Defaults**Example 7.22 — Omission defaults and caller preferences. $\mathcal{U}_1$**

```
add: func(a: int, b: int = 10) -> int: a + b
x: int | *5
omitted: add(1)                  // 11
supplied: add(1, x)              // 6
unknown: add(1, _)              // incomplete; b was supplied
```

An omission default belongs to the closure protocol. A supplied argument bypasses it and retains its own ordinary preference information. The default expression is checked against its parameter description in the declaration environment.

Example 7.23 — A defaulted name-only argument. $\mathcal{U}_1$

```
render: func(text: string, suffix!: string = "!") -> string:
  text + suffix
x: render("hello")              // "hello!"
y: render("hello", suffix: "?") // "hello?"
```

Defaulted required-label parameters may be omitted, while optional parameters retain absence semantics and have no omission default.

Example 7.24 — A configurable default is an explicit adapter. $\mathcal{U}_1$

```
raw: func(x: int) -> int: x
configured: func(x: int = 2) -> int: raw(x)
answer: configured()           // 2
```

The adapter supplies the protocol that accepts the empty packet. A contract mentioning that protocol requires an implementation possessing it. It does not mutate a previously supplied closure.

7.7 Open interfaces and partial application

An open signature describes a partial *protocol interface*. Elaborate its ellipsis to an existential row of further parameter declarations, retaining that witness with the function. Its callable packet domain is determined only when the required row is known. This is different from promising that every completion of the row can be omitted. $\mathcal{U}_1$ retains this row as an internal existential witness, just as it retains call-instantiation variables; explicit existential type packaging is a separate surface facility.

Example 7.25 — An open interface completed by an implementation. $\mathcal{U}_1$, *implicit protocol row*

```
T: func(a: int, ...) -> number
f: T & (func(a: int, b: int) -> int: a + b)
x: f(1, 2)           // 3
y: f(1)             // missing b
```

The row witness is completed by `b: int`. A closed one-argument capability admits an implementation with additional omissible parameters because its promised one-argument packets remain covered.

Example 7.26 — Chained partial application. $\mathcal{U}_1$

```
add3: func(a: int, b: int, c: int) -> int: a + b + c
f: add3(1, ...)
g: f(2, ...)
x: g(3)           // 6
y: add3(a: 1, ...)(2, 3) // 6
```

A partial closure stores bindings to original parameter slots. Supplied values are checked immediately; the body runs on completion. An unbound default is preserved and a bound argument suppresses that default.

Example 7.27 — A residual contract on a partial closure. $\mathcal{U}_1$

```
f: add3(1, ...)
f: func(int, int) -> int
x: f(2, 3)           // 6
```

The residual protocol is a normal callable contract. Normalizing the original slot bindings makes partial applications comparable independently of whether arguments were supplied by label or position.

7.8 Identity, captured values, and result combination

Use a closure descriptor (o, γ, β) : declaration origin o , captured free environment γ , and normalized partial bindings β . The origin survives copying; irrelevant ambient fields are absent from γ . Symbolic captures remain constraint variables.

Example 7.28 — Self-unification after copying and embedding. $\mathcal{U}_1$

```
x: {f: func(y: int) -> int: y}
a: x.f & x.f
b: x.f & {x}.f
c: x.f & (x & {g: 1}).f
r: [a(2), b(2), c(2)] // [2, 2, 2]
```

Example 7.29 — Equal source, unequal captures. $\mathcal{U}_1$

```
d: [for v in [1, 2] {func(y: int) -> int: y + v}]
e: d[0] & d[1]           // _|_: captures differ
```

For one origin, captured-environment constraints unify structurally. Distinct known origins denote distinct intensional values. This supplies idempotence without deciding equality of mathematical functions.

Example 7.30 — Combining result descriptions explicitly. $\mathcal{U}_1$

```
left: func(x: int) -> {a: int}: {a: x}
right: func(x: int) -> {b: int}: {b: x}
both: func(x: int) -> {a: int, b: int}: left(x) & right(x)
r: both(3)           // {a: 3, b: 3}
```

The body is a new implementation whose result is the CUE unification of the two result descriptions. Totality requires their intersection to be inhabited at every admitted input. A version with arbitrary possibly conflicting callbacks uses a checked conflict effect.

7.9 Validators and iteration**Example 7.31 — A predicate used as an ordinary constraint.** $\mathcal{U}_1$

```
positive: func(x: int) -> bool: x > 0
#Positive: {value: int, _valid: true & positive(value)}
n: (#Positive & {value: 7}).value // 7
bad: #Positive & {value: -1}      // _|_
```

A validator is a predicate on an existential subject. It can be expressed with existing struct machinery once predicate functions are available.

Example 7.32 — Portable foreign contract. $\mathcal{U}_1$ *with checked effects*

```
externF: extern func(int) -> int !bridge
// A native-range failure is a bridge outcome.
// A fractional input is outside the logical domain.
```

The bridge checks logical kind, representation, conversion, and returned values as specified by the source proposal [6]. The effect annotation records those failures; it introduces no additional conversion stage. Tool-level effects execute in an explicit sequencing context, while constraint unification operates on their descriptions.

Example 7.33 — Nested calls remain ordinary evaluation. $\mathcal{U}_1$

```
twice: func(n: int) -> int: n + n
value: twice(twice(twice(2))) // 16
```

The initial execution profile preserves the host cycle discipline. A structurally recursive extension can admit calls justified by a decreasing finite measure. This choice is independent of quantifier rank.

Example 7.34 — A structurally recursive fold. $\mathcal{U}_1$ *with structural recursion*

```
fold(A, B): func(step: func(B, A) -> B, seed: B, xs: [...A]) -> B: {
  if len(xs) == 0 {out: seed}
  if len(xs) > 0 {out: fold(step, step(seed, xs[0]), xs[1:])}
}.out
plus: func(x: int, y: int) -> int: x + y
sum: fold(plus, 0, [1, 2, 3]) // 6
```

Each recursive call reduces list length. The termination proof is independent of polymorphism; a nonrecursive profile may supply the same contract as a trusted primitive.

Chapter 8

Dependent descriptions and functions

8.1 Dependent contexts

A dependent context extends Γ by a value x satisfying a description A in that context:

$$\text{Env}(\Gamma, x : A) = \{(\rho, x) \mid \rho \in \text{Env}(\Gamma), x \in \llbracket A \rrbracket_\rho\}.$$

The existential and universal adjoints already apply to its projection. Dependence therefore changes the fibers over which quantification ranges, rather than introducing another interpretation of refinement.

A dependent function contract uses its actual argument in the result predicate:

$$\Pi_{x:A} B(x) = \{f \in F \mid \forall x \in A. G_{B(x)}(f, x)\}.$$

A dependent pair retains the witness:

$$\Sigma_{x:A} B(x) = \{(x, y) \mid x \in A, y \in B(x)\}.$$

Logical $\forall x B(x)$ constrains *one subject* at every x ; a dependent function instead relates inputs to results. Logical $\exists x B(x)$ projects away x ; a dependent pair retains it. The distinction is determined by the subject and the application or pairing relation.

Design rule. Within a dependent function signature, parameters enter scope from left to right. A result constraint can refer to all parameters. Default expressions can depend only on parameters whose binding is already established. The dependency graph determines evaluation order; it is independent of the source order of unrelated record declarations.

Example 8.1 — A directly dependent result. $\mathcal{U}_1 + \mathcal{D}$

```
successor: func(x: int) -> (x + 1): x + 1

// Equivalent pointwise specification:
successor(n in int): func(n) -> (n + 1)
```

The second form quantifies over input singletons. The first uses a dependent result predicate on the actual argument.

Example 8.2 — A dependent parameter constraint. $\mathcal{U}_1 + \mathcal{D}$

```
interval: func(lo: int, hi: int & >=lo) -> {
  low: lo
  high: hi
}: {low: lo, high: hi}

ok: interval(3, 7)           // {low: 3, high: 7}
bad: interval(7, 3)         // domain contradiction
```

The admitted domain is a relation on two arguments. It is not the Cartesian product of their marginal types.

8.2 Indexed finite sequences

The following aliases use the list validators supplied by CUE [4]:

```
import "list"

let Nat = int & >=0
Vec(A, n in Nat) = [...A] &
  list.MinItems(n) & list.MaxItems(n)
Fin(n in Nat) = int & >=0 & <n
```

Their reference semantics is

$$\llbracket \text{Vec}(A, n) \rrbracket = \{xs \in \text{List}(A) \mid |xs| = n\}, \quad \llbracket \text{Fin}(n) \rrbracket = \{0, \dots, n-1\}.$$

A symbolic index remains a constraint variable. The checker uses the length predicate and its lemmas; concrete validator execution is one implementation strategy for ground indices.

Example 8.3 — Length-preserving map. $\mathcal{U}_1 + \mathcal{D}$

```
map(A, B, n in Nat): func(
  g: func(A) -> B,
  xs: Vec(A, n),
) -> Vec(B, n): [for x in xs {g(x)}]
```

The universal n records a property of the input, rather than supplying a runtime integer. Its value is available as `len(xs)` when evaluation requires it.

Example 8.4 — Safe indexing, including the zero-length case. $\mathcal{U}_1 + \mathcal{D}$

```
index(A, n in Nat): func(xs: Vec(A, n), i: Fin(n)) -> A: xs[i]

x: index(["a", "b"], 1) // "b"
bad: index([], 0) // 0 is outside Fin(0)
```

For $n = 0$, the packet domain is empty. The generic implementation remains valid; there is no admitted out-of-range call.

Example 8.5 — Head with an explicit nonempty precondition. $\mathcal{U}_1 + \mathcal{D}$

```
head(A, n in int & >=1): func(xs: Vec(A, n)) -> A: xs[0]
first: head([4, 5]) // 4
bad: head([]) // no admissible instance
```

Example 8.6 — Appending adds lengths. $\mathcal{U}_1 + \mathcal{D}$

```
append(A, n in Nat, m in Nat): func(
  xs: Vec(A, n), ys: Vec(A, m),
) -> Vec(A, n + m): [for x in xs {x}, for y in ys {y}]

v: append([1, 2], [3]) // [1, 2, 3]
```

The proof combines the cardinalities of two concatenated finite comprehensions.

Example 8.7 — Zip rejects unequal lengths. $\mathcal{U}_1 + \mathcal{D}$

```
zip(A, B, n in Nat): func(xs: Vec(A, n), ys: Vec(B, n)) ->
  Vec([A, B], n): [for i, x in xs {[x, ys[i]]}]

z: zip([1, 2], ["a", "b"]) // [[1, "a"], [2, "b"]]
bad: zip([1], ["a", "b"]) // inconsistent length witness
```

The same existential instance of n must satisfy both input constraints at a call. Separate instances per argument would lose this invariant.

Example 8.8 — Split at a bounded position. $\mathcal{U}_1 + \mathcal{D}$

```
split(A, n in Nat, k in int & >=0 & <=n): func(
  xs: Vec(A, n), cut: k,
) -> {left: Vec(A, k), right: Vec(A, n - k)}: {
  left: xs[:cut]
  right: xs[cut:]
}
```

The runtime parameter `cut` reifies the index used by slicing. The endpoints $k = 0$ and $k = n$ are admitted.

Example 8.9 — Replication reifies its count. $\mathcal{U}_1 + \mathcal{D}$

```
replicate(A, n in Nat): func(x: A, count: n) -> Vec(A, n): [
  for i in list.Range(0, count, 1) {x}
]
r: replicate("x", 3)      // ["x", "x", "x"]
```

An erased type or value index cannot be inspected by a body merely because it appears in a contract. Here the actual count parameter supplies the needed runtime information. A checked library bridge can carry its declared allocation or representation effects; the finite reference range operation is total.

Example 8.10 — A dependent result with a hidden length. $\mathcal{S}_H + \mathcal{D}$

```
filterBounded(A, n in Nat): func(
  p: func(A) -> bool, xs: Vec(A, n),
) -> (exists (k in int & >=0 & <=n) Vec(A, k)):
  [for x in xs if p(x) {x}]
```

The existential witness can depend on the input and predicate. A $\mathcal{U}_1 + \mathcal{D}$ surface can express the same projected result using `[...A]` & `list.MaxItems(n)`. A witness-bearing version returns a record `{length: k, values: Vec(A,k)}`.

8.3 Matrix dimensions, including empty matrices

Retain dimensions as data when empty collections do not determine them:

```
Matrix(A, r in Nat, c in Nat) = {
  rows: r
  cols: c
  data: Vec(Vec(A, c), r)
}
```

A zero-row matrix still records its column count. This makes transposition and composition defined at all admitted dimensions.

Example 8.11 — Transpose with a precise shape. $\mathcal{U}_1 + \mathcal{D}$

```
transpose(A, r in Nat, c in Nat): func(m: Matrix(A, r, c)) ->
  Matrix(A, c, r): {
    rows: m.cols
    cols: m.rows
    data: [
      for j in list.Range(0, m.cols, 1) {
        [for row in m.data {row[j]}]
      }
    ]
  }
```

Every j is below the length c of every row. The outer length is c and the inner length is r , including either zero dimension.

Example 8.12 — Dot product with matched dimensions. $\mathcal{U}_1 + \mathcal{D}$; *finite fold primitive*

```

addNumber: func(x: number, y: number) -> number: x + y

dot(n in Nat): func(xs: Vec(number, n), ys: Vec(number, n)) ->
  number: fold(addNumber, 0, [for i, x in xs {x * ys[i]}])

```

Products and sums return `number`; arbitrary input refinements are not assumed closed under arithmetic.

Example 8.13 — Dimension-safe matrix multiplication. $\mathcal{U}_1 + \mathcal{D}$

```

matmul(r in Nat, k in Nat, c in Nat): func(
  x: Matrix(number, r, k), y: Matrix(number, k, c),
) -> Matrix(number, r, c): {
  rows: x.rows
  cols: y.cols
  data: [
    for row in x.data {
      [for j in list.Range(0, y.cols, 1) {
        dot(row, [for yrow in y.data {yrow[j]}])
      }]
    }
  ]
}

```

A single instance of k links the two inputs and justifies each dot product. The output carries the surviving dimensions r, c .

8.4 Protocol descriptions and refinement predicates**Example 8.14 — A request-indexed response.** $\mathcal{U}_1 + \mathcal{D}$ with a *checked service*

```

let Request = {kind: "lookup", key: string} |
  {kind: "size", items: [..._]}
Reply(q in Request) = {
  if q.kind == "lookup" {result: string}
  if q.kind == "size"   {result: int & >= 0}
}
request: extern func(q: Request) -> Reply(q) !service

```

The result predicate is evaluated in the request's environment. Replacing it by an independent union of responses would admit mismatched request/response pairs.

Example 8.15 — A relation between input and output records. $\mathcal{U}_1 + \mathcal{D}$

```

annotate(A): func(x: A, tag: string) -> {
  value: x
  label: tag
}: {value: x, label: tag}

```

The field labels differ from the parameter names. The result predicate states equality with the corresponding argument cells and retains the input/output correlation through subsequent record refinement.

Example 8.16 — Clamping to an argument-dependent interval. $\mathcal{U}_1 + \mathcal{D}$

```

clamp: func(lo: int, hi: int & >=lo, x: int) -> (
  int & >=lo & <=hi
): {
  if x < lo {out: lo}
  if x > hi {out: hi}
  if x >= lo && x <= hi {out: x}
}.out

bounded: clamp(0, 10, 14) // 10

```

The three branches partition the input domain. The dependency $hi \geq lo$ justifies each branch's interval postcondition. The result is established by linear arithmetic and branch refinement.

8.5 Proof fragments and erasure

Dependent specifications can be verified by a hierarchy of procedures: structural length reasoning, quantifier-free linear integer arithmetic, finite disjunction splitting, and explicit proof certificates for stronger predicates. Refinement-type inference provides a precedent for combining type structure with restricted automated implication checking [22].

The logical reference model admits every well-formed predicate. An executable profile grants a contract only after a sound derivation. A residual obligation retains its variables and scope. Runtime checks produce declared errors and do not turn a conditional check into a successful-call theorem.

The erasure property states that changing a proved annotation or choosing a type-instantiation derivation does not change the underlying closure and its runtime call packet. Value indices used computationally are ordinary arguments or fields, as in replication and matrices. This separates specification parameters from executable data without changing their shared set semantics.

Chapter 9

Existentials and modular reuse

9.1 A shared hidden representation

An existential record describes several fields using one representation witness:

```
#Counter: exists State {  
  zero: State  
  next: func(State) -> State  
  read: func(State) -> (int & >=0)  
}
```

Its meaning is $\bigcup_{S \in \mathcal{U}_n} \llbracket \text{Counter}(S) \rrbracket$. One S must make all three fields satisfy the interface. Replacing it with separate existential witnesses for the three fields loses the relationships that make composition type-correct.

This is the type-theoretic basis of abstract data types and modules identified by Mitchell and Plotkin [18]. In CUE, an additional distinction matters: existential projection is a logical operation, while *information hiding* is a restriction on the observations available at an interface.

9.2 Sealing and opening

A *seal* turns an implementation of an existential interface into an opaque public view. For an unbounded representation sort S , a private representation A is exposed through a fresh abstract copy

$$\widehat{A}_\kappa = \{(\kappa, a) \mid a \in A\},$$

where κ is a seal identity. Exported operations wrap and unwrap that copy through authorized adapters. The public existential witness is $\widehat{A}_\kappa$. Thus the set interpretation of the interface remains an existential union.

Public operations are exposed through opaque capability handles associated with the seal and the interface's declared export/alias graph. Their private code origins and captures are not observable through that view. Copying a sealed module preserves its seal and exported aliasing; explicitly constructing a fresh sealed module introduces a fresh seal.

Sealing and opening have the forms

```
seal Interface with (HiddenType = Representation) {  
  // implementation fields, checked with the private representation  
}  
  
open moduleValue as (AbstractType, View) {  
  // AbstractType is rigid; View has the opened interface.  
}
```

Opening introduces a local rigid type and preserves sharing across the view's field projections. The type cannot escape except under a corresponding existential package. $\mathcal{S}_1$ uses this operation at module boundaries; $\mathcal{S}_H$ admits it inside functions over first-class packages.

A transparent bounded existential still exposes every operation justified by its bound. A fully opaque initial sealing profile uses unbounded representation sorts and explicit exported operations. A bound such as `State: number` would expose numeric structure and entails a stronger observation-preservation condition on any representation change.

9.3 Two implementations and one client

Example 9.1 — An integer representation. $\mathcal{S}_1 + \mathcal{A}$

```
counterV1: seal #Counter with (State = int & >=0) {
  zero: 0
  next: func(x: int & >=0) -> (int & >=0): x + 1
  read: func(x: int & >=0) -> (int & >=0): x
}
```

Example 9.2 — A record representation. $\mathcal{S}_1 + \mathcal{A}$

```
let CounterRep = close({count: int & >=0})
counterV2: seal #Counter with (State = CounterRep) {
  zero: {count: 0}
  next: func(s: CounterRep) -> CounterRep:
    {count: s.count + 1}
  read: func(s: CounterRep) -> (int & >=0): s.count
}
```

The abstract carrier in each public view has no accessible count field or numeric representation. Its structure is the interface’s structure.

Example 9.3 — A client checked once. $\mathcal{S}_1 + \mathcal{A}$ with a rank-1 utility

```
useCounter(S): func(c: {
  zero: S
  next: func(S) -> S
  read: func(S) -> (int & >=0)
}) -> (int & >=0): c.read(c.next(c.next(c.zero)))

v1: (open counterV1 as (S, C) {out: useCounter[S](C)}).out
v2: (open counterV2 as (S, C) {out: useCounter[S](C)}).out
// Both observations are 2.
```

The client uses only the abstract operations. Its generic declaration is rank-1. The hidden type is opened at the module boundary and then supplied as a rigid instance.

Example 9.4 — A module law strengthened by universals. $\mathcal{S}_1 + \mathcal{D} + \mathcal{A}$

```
#CounterLawful: exists State {
  zero: State
  next: func(State) -> State
  read: func(State) -> (int & >=0)
  _zeroLaw: true & (read(zero) == 0)
  forall (s in State) {
    _stepLaw: true & (read(next(s)) == read(s) + 1)
  }
}
```

The law is a proof obligation on the module subject, not an instruction to enumerate all states. Both implementations discharge it by substitution into their bodies. The existential representation scope encloses the universal state-law scope.

9.4 Unification is an observation

Opaque values can participate in CUE unification. In the basic sealed model,

$$(\kappa, a) \& (\kappa, b) = \begin{cases} (\kappa, a), & a = b, \\ \perp, & a \neq b. \end{cases}$$

Consequently, a representation-independence theorem must preserve equality and conflict, not merely the outputs of named methods.

Definition 9.1 (Equality-respecting representation simulation). *For two representations A_1, A_2 , a simulation is a bijection $r : A_1 \rightarrow A_2$ on the entire carriers admitted by the public interface, together with corresponding exported capability handles, such that constants correspond, every public operation commutes with r , and public data results and effects agree. The carriers include the abstract states over which a client may quantify, as well as those reached by executing operations.*

For the counters, $r(n) = \{\text{count} : n\}$ is such a bijection. It preserves zero, successor, reading, and equality. The abstract public seal identities are related consistently across the two executions.

Extend r componentwise to data constructors and to sets of completions by direct image. A bijection preserves intersections, unions, emptiness, and singletons. Its transport of abstract type assignments induces bijections on the corresponding admissible predicate families.

Theorem 9.2 (Representation-independent replacement). *Assume an equality-respecting simulation, scope-preserving sealing and opening, and admissible type families and public operational inhabitants closed under the induced transport. Let the client's observations be confined to the declared public interface, ordinary data, unification, and the admitted effects. Replacing one sealed implementation by the related implementation preserves every client data observation and its success/conflict outcome.*

Proof. Interpret the client in the two related public environments. Constants and public operations preserve the relation by hypothesis. Reindexing and data constructors preserve it componentwise. Direct image along a bijection commutes with unions and intersections. Quantifiers range over corresponding assignments, so their unions and intersections commute with transport. Application preserves the relation by the operation simulation. Sealing preserves the public alias structure and hides implementation descriptors. Induction over the elaborated client derivation therefore relates its resulting completion predicates. At ordinary exported data, the relation is equality; emptiness also agrees. $\square$

The abstraction discipline is related to the classical logical-relations proof of representation independence [19, 10]. The explicit equality condition is required by CUE's observable meet operation.

A many-to-one abstraction relation requires additional treatment. If multiple private cache states represent the same logical state, their raw equality can be observable through unification. An interface may instead expose a certified quotient carrier and an equality congruence. Its sealing adapter then presents logical equivalence classes, and the theorem applies to the resulting public carrier. The implementation must realize that equality soundly.

9.5 The scope of the evolution guarantee

The theorem yields a checkable class of representation changes for which all admissible clients continue to work. Its hypotheses include behavior and observable equality. A signature that merely types `next` and `read` does not assert that `read(next(s)) = read(s)+1`; that equation is a separate interface law or simulation obligation.

For one-way evolution, an observation-preserving simulation can relate the old interface to a designated old-compatible view of the new implementation. New operations and states may be exposed through an extended view. If additional states are exposed through the old abstract type itself, preservation

also requires every old universally quantified obligation to hold on those states; agreement merely on previously executed states is insufficient. Effect guarantees and public serialization are part of the same simulation.

Here a *forward-compatible client* is a client whose old interface contract continues to hold when a later, related implementation is substituted. This is the usual backward-compatible replacement direction for the library. The quantified interface and simulation state the precise class of permitted future changes. Ordinary exposed structural schemas already support structural refinement and interface checks. Sealed existential witnesses add a generic *representation-independent* replacement guarantee. A private field alone does not abstract the representation of a public value whose numeric, record, or list structure remains observable.

9.6 First-class packages and existential placement

Example 9.5 — Heterogeneous values with their own operations. $\mathcal{S}_H + \mathcal{A}$

```
#Showable: exists A {
  value: A
  show: func(A) -> string
}
p: seal #Showable with (A = int) {
  value: 7
  show: func(x: int) -> string: "\({x})"
}
q: seal #Showable with (A = string) {
  value: "hello"
  show: func(x: string) -> string: x
}
items: [p, q]
```

Each package has its own type witness and seal. A list of packages differs from one package containing a homogeneous list.

Example 9.6 — Eliminating a first-class package. $\mathcal{S}_H + \mathcal{A}$

```
describe: func(p: #Showable) -> string:
  (open p as (A, P) {out: P.show(P.value)}).out
text: map(describe, items) // ["7", "hello"]
```

At a sealed interface, projections are mediated by the opened abstract view. The checker does not independently instantiate the hidden type for show and value.

Example 9.7 — Sharing two values under one witness. $\mathcal{S}_H + \mathcal{A}$

```
#ComparablePair: exists A {
  left: A
  right: A
  equal: func(A, A) -> bool
}
compare: func(p: #ComparablePair) -> bool:
  (open p as (A, P) {out: P.equal(P.left, P.right)}).out
```

Separate existential packages for the two operands would require an explicit sharing certificate before one comparator could consume both.

Example 9.8 — An abstract codec interface. $\mathcal{S}_1 + \mathcal{A}$; *checked effects*

```
#CodecModule: exists Value {
  encode: func(Value) -> bytes
  decode: func(bytes) -> Value !decode
}
```

A round-trip law requires decoding an encoded value to return the corresponding abstract value. Persisted wire compatibility additionally constrains the chosen byte representation; hiding an in-memory representation does not silently version an external protocol.

9.7 Mixed prefixes and generic modules

The order of module binders expresses representation dependence:

```
module(A): {
  exists State
  empty: State
  push: func(A, State) -> State
}
```

The logical order is $\forall A \exists State$. It allows the hidden semantic state type to depend on A while constraining one uniform module subject. A standard witness is $State = \text{List}(A)$, with empty list and a generic cons operation. It differs from $\exists State \forall A$, which chooses one semantic state type before the element type varies.

When different *runtime* modules are required for different arguments, an ordinary module-producing function supplies the module subject. Quantifiers specify its interface; they do not themselves allocate a record at each type assignment. This keeps generativity, runtime allocation, and logical dependence separate and explicit.

Chapter 10

Checking, execution, and formal obligations

10.1 Judgments and rigid scopes

Use a constraint context Γ with rigid variables, flexible existential variables, and a scope level for each variable. The principal judgments are

$$\Gamma \vdash e \Rightarrow T, \quad \Gamma \vdash e \Leftarrow T, \quad \Gamma \models T \leq U.$$

They respectively synthesize a description, check an expression against a description, and establish semantic inclusion. The checker returns a proof, a refutation with a witness, or a residual obligation.

Universal introduction checks one expression under an arbitrary rigid variable:

$$\frac{\Gamma, A : \text{Type}(n) \vdash e \Leftarrow T \quad A \notin \text{FV}(\Gamma, e_{\text{run}})}{\Gamma \vdash e \Leftarrow \forall A T}.$$

The side condition refers to erased runtime code; A may occur in checked annotations. A bounded introduction adds its bound as an assumption. The same runtime expression must be valid at every assignment.

Universal elimination substitutes a well-sorted admissible type. Inference may introduce a fresh flexible type and solve its constraints. Different uses obtain different instantiation variables; the declared universal remains unchanged. Existential introduction supplies a witness. Elimination opens one shared witness in a fresh rigid scope, and the result must respect its escape condition.

These scope rules have implementation precedents in higher-rank bidirectional checking and quantified logic programming [20, 21]. A flexible witness may depend only on the rigid variables available at its introduction site. The solver never resolves an outer existential by referring to a later universal.

10.2 Functions and declaration conjunction

A lambda rule checks the body on arbitrary admitted packets and proves its successful result predicate. In the nonrecursive core, evaluation is structurally bounded; primitives carry checked contracts. A structural-recursion profile adds termination certificates.

Conjunction accumulates predicates on the same subject. In particular,

$$\frac{\Gamma \vdash e \Leftarrow T \quad \Gamma \vdash e \Leftarrow U}{\Gamma \vdash e \Leftarrow T \wedge U}$$

concerns one expression, not a synthesis of two bodies. Application uses the correlation-preserving arrow clauses applicable to the actual packet. Subsumption uses semantic inclusion. Existential calls can propagate constraints on their argument and result cells without granting an unproved universal capability to the selected implementation.

Theorem 10.1 (Soundness of the reference rules). *Assume sound primitive contracts and implication procedures, a scope-correct elaboration, and the termination obligations of the selected execution profile. If the reference checker establishes $\Gamma \vdash e \Leftarrow T$, then every admitted environment gives the erased expression an inhabitant of $\llbracket T \rrbracket$. For a checked successful arrow, every admitted call packet produces a nonempty result description contained in its codomain. Checked effects are limited to the declared outcome set.*

Proof. Induct on the checking derivation. Base cases use the primitive hypotheses. Conjunction and disjunction use set membership. Subsumption uses inclusion. The lambda case is the universal

packet obligation in equation (4.1); the application case instantiates it. Universal introduction ranges over arbitrary rigid assignments while preserving erased code; elimination specializes the intersection. Existential rules choose or open a witness while preserving its scope. Termination and primitive outcome obligations establish the successful or admitted-effect cases. $\square$

Sound checking permits annotations and residual obligations; it need not be complete.

10.3 Adequate symbolic evaluation

Let $\text{Comp}(E)$ denote successful complete observations of an elaborated expression, and let $N(E)$ be the evaluator's retained symbolic graph. The adequacy target is

$$\llbracket N(E) \rrbracket = \text{Comp}(E)$$

for the selected reference fragment, with preference and failure observations recorded separately. A conservative abstract interpreter may instead retain an over-approximation for propagation; its final conformance judgments must still be justified independently.

Fresh call activations copy parameter cells and preserve closure captures. Function application does not invent implementations to satisfy a codomain. A call on an unspecified implementation can remain incomplete. A contradiction is established only when the retained constraints have no admissible completion, or a concrete computation produces a specified failure.

Memoization keys include closure identity, bound packets, and the relevant resolved environment. Residual obligations are retained with their dependency links. A later refinement invalidates stale approximations or further restricts them. Cached concrete results can only survive or conflict under the refinement theorem.

10.4 Decision fragments

A useful implementation combines finite Boolean type structure, a sound semantic-subtyping procedure, finite sets of polymorphic instances, and a restricted solver for dependent predicates. Castagna–Laurent–Nguyen give a sound terminating reconstruction algorithm for a particular first-order polymorphic set-theoretic calculus [16]. That result motivates $\mathcal{U}_1$; its hypotheses and arrow interpretation remain part of its own calculus.

An implementation reports *established*, *refuted*, or *unresolved*. Treating an unresolved implication as false would turn later proof discovery into an apparent repair of bottom. Treating it as true would lose the universal guarantee. The three outcomes therefore belong to the checker contract. Chapter 11 specifies a finite local kernel, persistent proof obligations, and demand-driven concrete validation for that contract.

Chapter 11

Refinement-stable normalization and checking

11.1 Hypotheses, refutation, and observation

A declaration contributes a hypothesis to a retained constraint store. Accepting that store for further unification does not certify that it has a completion. Denotational satisfiability and operational refutability are distinct judgments:

$$\llbracket P \rrbracket \neq \emptyset, \quad P \vdash_{\mathcal{K}} \perp.$$

Here $\mathcal{K}$ is a specified, sound proof kernel. Soundness requires $P \vdash_{\mathcal{K}} \perp \implies \llbracket P \rrbracket = \emptyset$; the converse is not required. An unrefuted empty predicate remains denotationally empty. Its residual representation records the hypotheses whose incompatibility has yet to be established.

Example 11.1 — An empty arithmetic relation retained as hypotheses. *Base fragment*

```
a: int & b * 5
b: a - 1
```

The equations imply $a = 5(a - 1)$, hence $a = 5/4$, contradicting $a \in \mathbb{Z}$. A kernel that evaluates multiplication and subtraction only when their operands are concrete retains these equations. It need not perform symbolic elimination. After adding $a: m$ for any concrete integer m , it computes $b = m - 1$ and checks $m = 5(m - 1)$. That concrete instance is refuted. For example, $a: 0$ yields $b: -1$ and the conflict $0 = -5$. Refuting one instance constrains that instance; it is not a proof that an independently instantiated schema has no other witnesses.

Example 11.2 — A local bound contradiction. *Base fragment*

```
x: <0
y: x & >0
```

Copying the constraint on x into y exposes the empty interval $(-\infty, 0) \cap (0, \infty)$. A local bound rule refutes it without choosing a concrete atom. This is as stable under refinement as the arithmetic refutation; it is simply cheaper to discover.

The distinction between suspended expressions and demanded concrete evaluation already appears in CUE’s treatment of reference cycles [24]. For the extension, the implementation contract specifies which local rules run and when observations demand their residual obligations. The mathematical meaning of a declaration does not depend on the amount of search performed.

11.2 Stable reductions

Write $P \simeq Q$ when two residual predicates have the same denotation on the same exposed interface, with their local witnesses projected according to their scopes. A normalizing step must satisfy

$$P \longrightarrow Q \implies P \simeq Q. \quad (11.1)$$

Every subsequent conjunct R then gives

$$P \wedge R \simeq Q \wedge R.$$

This is the extension-stability condition. A step introducing fresh local variables must preserve their projection and the free-variable interface. Steps under quantifiers use equivalence in that predicate fiber, followed by the quantifier; they preserve guards and dependencies.

Two useful special cases are consequence propagation and certified refutation:

$$\frac{P \vdash_K C}{P \longrightarrow P \wedge C}, \quad \frac{P \vdash_K \perp}{P \longrightarrow \perp}.$$

Neither permits deleting an unresolved premise. A primitive equality such as $z: x + y$, with $x: 2$ and $y: 3$, can be replaced by $z: 5$ while retaining the input bindings. A conjectured solution to an existential constraint cannot be committed unless equivalence, rather than mere satisfiability of one branch, justifies the commitment.

Example 11.3 — A concrete result with a residual guard. *Base fragment*

```
x: int
r: 3 & (x + 1)
```

The result cell may carry the candidate atom 3 , but the equality $3 = x + 1$ remains active. Adding $x: 2$ validates it; adding $x: 5$ refutes it. Replacing the whole expression by an unconditional 3 would lose a constraint. A backward rule deriving $x: 2$ is also sound if the primitive's integer semantics justifies it; the minimal kernel need not include that rule.

Monotonicity constrains admissible decisions, not the completeness of the search for them. Sound symbolic arithmetic, for example, could refute the first example immediately. A terminating implementation instead saturates a declared finite rule fragment and retains the remaining program. This makes the criterion precise: every committed decision has persistent evidence; computational effort is a separate resource policy.

Semantic entailment from an empty predicate is vacuous. It therefore cannot, by itself, justify materializing an arbitrary concrete result. Materialization requires an actual primitive-evaluation or constructor derivation, together with any still-active guards. An inconsistency proof produces bottom, not invented data.

11.3 Exact residuals and approximate propagation

An implementation state is

$$S = (\Sigma, \Phi, K, W).$$

Σ is the binding and sharing graph, including quantifier levels and subjects. Φ contains the exact residual predicates and unevaluated expressions. K is a graph of checked consequences with their supporting premises. W is a worklist of enabled propagation, evaluation, and checking tasks. Only Σ and Φ determine the state's denotation; K and W record evidence and scheduling.

An approximate descriptor d_x for a cell x has a concretization $\gamma(d_x) \subseteq V$ satisfying

$$\text{proj}_x \llbracket S \rrbracket \subseteq \gamma(d_x).$$

Shapes, primitive kinds, explicit bounds, and finite constructor alternatives are useful such descriptors. Their role is abstract interpretation [25]. Intersecting two sound upper approximations is sound, and an empty upper approximation proves a conflict. A nonempty upper approximation proves neither existence of a completion nor realizability of a function.

The exact predicates remain in Φ . Replacing them by the approximation alone would enlarge their denotation. In particular, a function's parameter descriptor is not substituted for a shared existential type variable. Bounds on that variable stay correlated with the function and with every use of the variable. Each certificate records its scope and assumptions. A certificate valid under a guard G licenses the consequence $G \Rightarrow C$, not the unconditional C . An implication proved from other interface hypotheses is a conditional proof; those hypotheses remain attached until independently discharged or preserved in the exported interface.

11.4 A finite local kernel

The default kernel operates on a shared graph and retains disjunctions without expanding them into a Cartesian product. It provides the following rules.

Rule family	Evidence and action
Atoms and kinds	Compare concrete atoms exactly; intersect primitive-kind sets; reject a required cell with incompatible kinds.
Bounds	Intersect explicitly available numeric bounds, retaining strictness and integer membership; detect an empty interval.
Records and lists	Propagate constraints to shared fields and known positions. Respect field presence, closedness, and list tails. An optional impossible field forces absence rather than rejecting an otherwise valid record.
Equality	Maintain equivalence of cells established by the elaboration. Propagate constants across those classes; preserve local witness scopes.
Ground primitives	Evaluate pure operations when their required operands are concrete; retain a suspended operation otherwise.
Boolean structure	Discharge a conjunct with a proof. Remove a disjunct only with a refutation of that branch. Refute a disjunction only when all its arms are refuted.
Quantified clauses	Combine equal-scope universal clauses pointwise; instantiate at explicit or demanded admissible arguments; preserve the original clause.
Function obligations	Decompose annotated arrows and structural bodies using sound introduction rules; retain unresolved coverage, outcome, and dependent predicate obligations.

The record and list rules operate on the host elaborator’s presence constraints, not merely on displayed labels. They retain its distinctions between open, closed, required, optional, hidden, and definition fields. Constructor cycles likewise use the selected execution discipline; equality propagation does not impose an unrelated finite-tree occurs check on recursive descriptions.

A shape mismatch is refutable when the conflicting shape is required in every remaining possibility. For a function contract, refuting its result shape also requires an admitted call. A successful arrow with empty domain imposes no result obligation. A possibly empty guarded domain is therefore retained until its relevant emptiness or inhabitation is established.

11.5 Checking goals are residual constraints

Formation, type checking, and evaluation share Σ , Φ , and K . Formation checks binding, sort, universe level, and profile restrictions. Checking an implementation generates predicates on that implementation; normalization processes them using the same kernel as other constraints. Normalization, materialization, and certification select which obligations are demanded. They share the monotone store-and-proof discipline of constraint programming [26].

Distinguish three questions:

$$\llbracket S \rrbracket \subseteq \llbracket T \rrbracket, \quad \llbracket S \rrbracket \cap \llbracket T \rrbracket = \emptyset, \quad f \in \llbracket T \rrbracket.$$

They ask for inclusion, disjointness, and membership of a particular implementation. Failure to prove the first is not evidence for the second. An implementation known only to satisfy S may still satisfy a narrower T .

Example 11.4 — A broad approximation is not a contradiction. $\mathcal{U}_1$

```

f: func(x: number) -> number: 1
f: func(number) -> int           // provable from the body

unknown: func(number) -> number
unknown: func(number) -> int     // a nonempty refinement

```

The checker uses the body of `f`, rather than treating its declared result upper bound as an exact behavior. The second pair constrains an unspecified implementation and leaves its witness unresolved.

For a known lambda and an obligation $f \in A \Rightarrow B$, introduce an arbitrary rigid packet $p \in A$, form a fresh activation, and check the body. A certificate establishes termination, permitted outcomes, and the required nonempty result predicate for every such packet. Structural constructors, identity, certified primitive applications, and finite comprehensions supply cheap introduction rules. Unsupported arithmetic facts become residual goals.

For example, the kernel can prove arrow inclusion by the rule

$$\frac{\Gamma \vdash A \leq C \quad \Gamma \vdash D \leq B}{\Gamma \vdash (C \Rightarrow D) \leq (A \Rightarrow B)}.$$

For a lambda it can instead prove the stronger property directly from the body, even when a previously inferred upper bound gives no useful inclusion. These are alternative proof procedures for the same target predicate.

Proof-search premises and program constraints occupy different parts of the state. A sufficient rule's undecided premises remain tasks in W ; they are not conjoined into Φ as additional requirements unless they are equivalent to the original target or are themselves entailed. Otherwise an incomplete proof attempt would strengthen the program and discard valid implementations. For a universal goal, a residual premise obtained under a rigid A likewise retains its universal guard. Dropping that scope would turn an arbitrary-case obligation into a satisfiable-instance search.

The target obligation may be used as a hypothesis to constrain calls, but not as its own evidence of implementation conformance. Proof dependencies must be well founded. An annotated callback parameter is a legitimate assumption of the caller's conditional proof; the caller's implementation must preserve the obligation to validate the actual callback. Mutually assuming two implementations' annotations is not a discharge of either obligation.

Example 11.5 — Concrete counterexamples to implementation contracts. $\mathcal{U}_1$

```

wrongResult: func(int) -> string
wrongResult: func(x: int) -> int: x

wrongDomain: func(number) -> string
wrongDomain: func(x: int) -> string: "ok"

```

The input `0` refutes the first contract. The input `1.5` is admitted by the second contract and rejected by its fixed implementation protocol. These are finite, stable witnesses, not failed attempts to prove subtyping.

11.6 Universal and existential proof search

A universal clause is stored as a scoped template. It has two complementary uses: *propagation* instantiates a hypothesis at a demanded argument; *certification* checks one implementation under arbitrary rigid binders. A finite set of successful tests supports propagation at those instances; it does not certify the universal clause.

Example 11.6 — Uniform proofs and finite refutations. $\mathcal{U}_1$

```
id(A): func(x: A) -> A: x      // one structural proof
bad(A): func(x: A) -> A: 0     // refuted at A = 1, x = 1
```

For `id`, the result is the input cell, so the proof uses the same arbitrary A . For `bad`, the singleton instance gives incompatible atoms. A kernel may search a small, fixed vocabulary of witness types and values. Failure of that search leaves the obligation unresolved.

Example 11.7 — A symbolic inconsistency exposed by one universal instance. $\mathcal{U}_1$

```
f(A): func(A) -> A
r: f(3) & string
```

The instance $A = \{3\}$ entails that a successful result is 3. Together with the required result constraint, this refutes the configuration without selecting an implementation for `f`. The universal hypothesis is retained throughout; it is not replaced by this one instance.

Existential introduction instead records an unresolved witness and its constraints. A witness assignment can replace that variable only when the substitution is an exact solution or when projection justifies the elimination. A search heuristic that finds one of several possible witnesses retains the alternatives as a disjunction or a residual existential. It cannot choose one arbitrarily in an order-independent configuration.

Example 11.8 — A flexible witness must remain refinable. $\mathcal{S}_1$

```
x: exists (n in 1 | 2) {value: n}
x: {value: 2}
```

The first declaration admits both witnesses. Committing to $n = 1$ would introduce an error repaired by choosing a different solution. Retaining the existential allows the second declaration to remove the first alternative.

Rigid parameters are arbitrary but not concrete. Their operations remain symbolic except where the kernel proves a uniform result. Their scope levels prevent an outer existential from being solved by a later universal. Binder normalization uses the laws of chapter 3, including the empty-range side condition for universal floating.

11.7 Counterexamples and remaining environments

A failed candidate is not necessarily a failed configuration. Suppose a guarded universal is $\forall a \in D_\rho P(\rho, a)$ and a concrete d has been identified. An unconditional refutation requires evidence, under the current remaining environments, that $d \in D_\rho$ and $\neg P(\rho, d)$. If failure holds only for a subset H of those environments, the sound consequence is exclusion of H . Other environments remain possible.

Example 11.9 — A refinable argument restricts a call, not a universal domain. $\mathcal{U}_1$

```
x: number
f: func(x) -> string
f: func(n: int) -> string: "ok"

x: int
```

The singleton argument contract on x excludes noninteger assignments. A trial with $x = 1.5$ refutes that assignment, not every assignment of x . Refining it to `int` cannot repair bottom, because the earlier joint constraint was never refuted. The same rule applies to an existential type variable X in $f \in X \Rightarrow B$: a rejected packet removes choices of X that contain it, rather than substituting the current upper bound for X .

This distinction also governs branch pruning. A counterexample to one candidate implementation or one union arm does not establish that their existential union is empty. Branch-local certificates keep their guards until all alternatives are excluded or a remaining branch is forced.

11.8 Demanded calls and concrete observations

A demand identifies an observation and its dependency graph. Requiring a concrete data result wakes its primitive operations, equalities, field-presence checks, and result constraints. An operation producing the wrong concrete atom or shape yields a refutation of that demanded instance.

The evaluator returns one of

$$\text{Conflict}(\pi), \quad \text{Value}(v; S'), \quad \text{Blocked}(S').$$

π is a checked refutation. In $\text{Value}(v; S')$, v has a concrete constructor or evaluation derivation and S' retains its guards and all unresolved declarations. It is a concrete observation *with a residual program*, not a certificate of global satisfiability. A data export declared complete must discharge its demanded validation obligations; otherwise it reports incompleteness. A universally specified module declared verified must also discharge its universal conformance obligations.

Example 11.10 — Local execution and universal verification. $\mathcal{U}_1$

```
externalArithmetic: extern func(int) -> int !bridge
f: func(x: int) -> int: externalArithmetic(x)
r: f(0)
```

A terminating, correctly typed observation of `externalArithmetic(0)` may establish the particular result of `r`, with the remaining contract still in the store. It does not establish correct termination and results at every integer. The foreign operation is evaluated only at its declared execution boundary. Certification of the pure arrow requires independent evidence of the universal property; a checked foreign arrow instead records the permitted failures as in chapter 4.

A complete closure descriptor is concrete as an implementation identity, but membership in an arbitrary universal type can still require an unresolved proof. Concrete-data validation and universal implementation certification have different completeness requirements.

Proposition 11.1 (Concrete-data completeness of the ground kernel). *Consider a finite, ground validation graph whose leaves are decidable primitive data predicates, whose operations terminate on the supplied concrete arguments, and whose required structure and labels are fixed. Fair evaluation and exact local validation either establish every demanded constraint or produce a finite refutation. No inconsistent assignment in this fragment is reported complete.*

Proof. Evaluate each ready primitive with exact arguments and compare its result with all required constraints. The graph is finite, and the stated evaluation hypothesis makes every demanded operation decidable. A false leaf or conflicting result supplies a refutation; if every check succeeds, their finite conjunction has been validated at the supplied assignment. $\square$

For an already assigned cyclic graph, validation checks its equations against that assignment without recursively reevaluating the cycle. Ungrounded cycles and hidden quantified obligations remain residual unless another rule decides them. The proposition is a completeness guarantee for checking supplied data, not for finding its witnesses.

11.9 Incremental algorithm

Each declaration contributes scoped constraint nodes and enabled tasks. The normalizer is a worklist interpreter with checked rule instances:

```

refine(state, declarations, demands, resources):
  elaborate_and_conjoin(state, declarations)
  enqueue_new_dependencies(state, demands)
  while an enabled task has an available resource ticket:
    task = choose_task_deterministically(state)
    result = local_kernel(task, state)
    match result:
      equivalent_rewrite(replacement, proof):
        verify(proof); install_preserving_scopes(replacement)
      consequence(fact, proof):
        verify(proof); conjoin_and_record(fact, proof)
      refutation(guard, proof):
        verify(proof); exclude_guarded_case(guard, proof)
      blocked(dependencies):
        retain_and_watch(task, dependencies)
    enqueue_tasks_whose_inputs_gained_information(state)
  return state_with_all_residuals_and_certificates(state)

```

A proof must be checked before its consequence is committed. A refuted unconditional case becomes bottom; a guarded case is eliminated only within its guard. Unchanged blocked tasks sleep until a relevant premise is refined. The worklist does not repeatedly unfold unresolved arithmetic cycles.

Memoization uses stable subject identities, binder identities, explicit instantiations, and supporting premises. A proved consequence stays valid under additional conjunction. An unresolved cache entry is merely a suspended task, not a cached negative answer. A cached concrete atom keeps its validation edges.

Resource tickets bound speculative branch splitting, witness search, and type instantiation separately from ground evaluation. Tickets are attached to stable tasks or origins; adding an unrelated declaration does not withdraw earlier certificates or steal a completed task's proof. An incremental run preserves all earlier evidence. A fresh run can replay that evidence independently of its search budget. File-order independence is a property of the retained predicates and certificates, not a claim that different search budgets produce identical printed residual graphs.

11.10 Preservation and persistence

Theorem 11.2 (Sound incremental normalization). *Assume scope-correct elaboration, sound local certificates, exact ground primitives, and retention of residual guards. For every finite normalization sequence $S \longrightarrow^* S'$:*

1. $\llbracket S \rrbracket = \llbracket S' \rrbracket$ on the exposed interface;
2. every recorded refutation is sound, and remains sound after additional conjunction in its recorded scope;
3. every concrete observation retains its validation dependencies, so a later refinement can preserve its atom or refute that guarded observation, but cannot justify a different atom for the same retained observation;
4. no universal conformance certificate is obtained solely by enumerating successful instances or by assuming the obligation being certified.

Proof. Induct on the transitions. Equivalent rewrites use equation (11.1). Adding a certified consequence preserves the predicate. A guarded refutation removes only an impossible case. Scope-preserving quantifier steps use reindexing and the adjunction rules. Ground execution uses the primitive hypothesis and retains the equations connecting its result to its inputs. Conjoining another predicate preserves

every such equivalence and implication. A concrete atom is a singleton, so a stronger result constraint can only retain it or be disjoint from it. The certificate checker enforces the universal introduction and dependency conditions on every discharged goal. $\square$

The theorem is about finite evidence and residual programs. It permits a satisfiable store, an unrefuted empty store, and a store awaiting an abstract implementation to remain incomplete. It does not identify any of them with a successful closed implementation. Defaults, comprehension elaboration, and external effects retain their specified observation boundaries.

11.11 Cost and profile-specific decision fragments

For the finite propagation core, fix the graph and the available descriptor vocabulary. Let n be its number of cells and obligation nodes, m its dependency edges, h the maximum number of strict descriptor refinements per node, and c a bound on one transfer or certificate check. A worklist that rechecks a node only when an input changes has the bound

$$O(m \alpha(n) + (n + m)h c),$$

with union–find cost α for equality classes. This bound counts graph operations; exact arithmetic contributes its bit cost to c . Finite bound endpoints and shape flags give a finite h . Ground calls and newly requested instances extend the graph and are charged separately. Unrestricted interval strengthening or Boolean distribution does not satisfy the finite-vocabulary hypothesis.

A small nonbranching kernel is therefore cheap, even though arbitrary semantic inclusion is not its decision problem. Optional stronger procedures may return certificates, refine residuals, or exhaust their budget. Exhaustion produces a residual obligation, never a contradiction or a conformance certificate.

Profile	Default checking organization	Exact fragment and residual boundary
$\mathcal{U}_1$	Check an outer universal once with rigid type variables; create flexible instances at uses; propagate quantifier-free clause constraints.	Annotated, acyclic structural terms over a finite base-type algebra admit a terminating structural checker. Refinement predicates and unresolved semantic inclusions remain goals.
$\mathcal{S}_1$	The same engine with existential module witnesses, scope levels, and escape checks at fixed boundaries.	Explicit witnesses and openings reduce to checks in the selected monomorphic fragment. Arbitrary witness synthesis and module laws remain residual.
$\mathcal{S}_H$	Bidirectional checking with explicit introduction and elimination at nested quantified positions.	A fixed predicative annotated fragment can have decidable checking; arbitrary quantified Boolean subtyping is not inferred from that fact.
$\mathcal{D}$	Generate value-indexed arithmetic, equality, and shape goals in the same store.	Ground validation is exact. A selected decidable arithmetic theory can discharge more goals; unsupported predicates stay symbolic.

Decidability belongs to a precisely delimited judgment and predicate language, not to rank alone. Rank-1 set-theoretic reconstruction and annotated higher-rank bidirectional checking both have terminating algorithms for particular calculi [16, 20]. Their restrictions delimit useful components, rather than deciding all predicates expressible in the present language.

Chapter 12

Feature correspondence and related designs

12.1 The comparison baseline

The July 2026 function proposal describes opaque function values, packet binding, defaults, open signatures, and partial applications. The September theory addendum analyzes a covariantly tightened call-constraint interpretation and pointwise conjunction of bodies. It separately proposes coverage obligations and compatibility checking [5, 6].

The present calculus expresses call restrictions existentially and reusable capabilities universally. Its set lattice therefore includes both relations on actual call cells and predicates on implementations. Their different variance follows from their quantifiers.

12.2 Feature coverage

Feature	Interpretation or construction in quantified CUE
Legacy records, lists, unions, bounds	Unchanged data predicates and legacy elaboration; definition 5.2.
Definitions, closedness, optional and required fields	Retained field-presence and label constraints; quantifiers do not add fields.
References, embedding, comprehensions	Existing copying plus capture-avoiding binder and witness rebinding.
Literal bodies and inferred result constraints	Operational closures with a checked declared or inferred result predicate.
Positional and labeled arguments	Complete call packets and a fixed binder.
Required, optional, hidden, and anonymous parameters	Presence alternatives, package-qualified labels, and positional slots.
Body-only aliases	Lexical aliases with no public packet label.
Parameter order and uniqueness	Protocol well-formedness before value unification.
Parameter attributes	Retained metadata, interpreted only by the relevant declared extension.
Omission defaults	Implementation-owned packet completion; supplied arguments bypass them.
Caller disjunction defaults	Existing preference rules on the supplied value.
Default contributed by another declaration	Explicit defaulted adapter or a refinable description of a yet-unselected implementation protocol.
Label contributed to an existing closure	Explicit label adapter; fresh builtins may receive their generated binder at construction.
Function types and intersections	Sets of operational inhabitants satisfying all universal arrow clauses.
Open signatures	Existential protocol rows retained until the remaining required packet shape is known.
Higher-order parameters and results	Ordinary arrows; universals specify reuse across types.
Partial application	A new closure with normalized original-slot bindings and a residual contract.
Constraint on a partial value	An ordinary constraint on that residual closure.
Distinct bodies combined pointwise	An explicit body that calls both and unifies their result descriptions.
Self-unification and captures	Stable origins and structurally constrained captured environments.
Validators	Predicates on an existential subject, definable with existing structs.
List-based variadic use	One list-valued packet component, with a generic element type.

Feature	Interpretation or construction in quantified CUE
Nonrecursive execution and cycles	The legacy execution profile; structural recursion is a separate opt-in policy.
Builtins and foreign calls	Typed descriptors with an explicit admitted effect set and the existing bridge.
Compatibility checking	Packet-domain inclusion, output/effect refinement, and module simulations where abstraction applies.
Export, formatting, and round trips	Source export retains binder scopes, closure environments, and bound slots; data export retains its existing supported values.
Memoization and dependency tooling	Scope-preserving graph traversal and dependency-aware caching.

Experimental operations that transform a closure elaborate to explicit transformations. Contract intersection asserts properties of the original closure.

12.3 Three explicit translations

Example 12.1 — Supplying a label. $\mathcal{U}_1$

```
raw: func(_~n: int) -> int: n + 1
named: func(x: int) -> int: raw(x)
a: named(x: 2)           // 3
```

The adapter’s packet language admits x . Raw positional behavior remains unchanged.

Example 12.2 — Restricting public calls across files. $\mathcal{U}_1$

```
raw: func(x: number) -> string: "\(\x)"
call: {arg: number, result: raw(arg)}
call: {arg: int, result: string}

ok: (call & {arg: 7}).result // "7"
bad: (call & {arg: 1.5}).result // _|_
```

This is ordinary CUE instantiation and projection. The argument restriction is an existential predicate, while `raw`’s capability remains universal.

Example 12.3 — Keeping late contract failures explicit. $\mathcal{U}_1$ with checked effects

```
checked: func(x: int) -> string !constraint:
  checkedResult(string, legacyBody(x))
```

Here `checkedResult( $T$ ,  $e$ )` is a reference adapter operation: evaluate e , validate its result against T , and return either a successful result or the named constraint-failure outcome. The adapter’s contract describes late failures honestly. A successful arrow is granted only after their absence is established.

Proposition 12.1 (Contract intersection). *Under the universal arrow interpretation, conjoining a contract with a concrete closure cannot change its call protocol or result. It preserves the singleton closure or makes its intersection empty.*

Proof. A subset of the singleton $\{f\}$ is either $\{f\}$ or empty. □

Consequently, constructing a label/default adapter and asserting a contract are different operations. Keeping that distinction also preserves associativity. A full experimental compatibility layer can implement its original invocation relation as a checked adapter, while translating its public guarantees into the appropriate quantified predicates.

12.4 Modular guarantees

Universal constraints express a statement the call-restriction interpretation does not express by itself: one selected implementation handles every admitted instance. This supports checking map, polymorphic callbacks, and bounded refinement-preserving utilities without seeing their eventual calls. The intersection algebra preserves input/output correlations. Dependent universals state relationships across argument and result values. Existential seals support representation-independent replacement under an explicit theorem.

Untyped callbacks and struct encodings can express many of the same computations. Quantified descriptions additionally preserve their relationships as constraints.

12.5 CDuce and semantic subtyping

Frisch, Castagna, and Benzaken develop semantic subtyping with function and Boolean types; Castagna and Xu extend its foundation to polymorphic types and convex models [11, 12]. CDuce’s polymorphic calculi combine instantiation with intersections and preserve input/output correlations [13, 14]. These supply close precedents for the arrow and instantiation layers here.

The current proposal adds explicit quantifiers to the general CUE predicate language. Its total-success and effect-qualified arrows refer to an operational inhabitant model. A subtyping algorithm proved for a different arrow model must be revalidated for the selected reference fragment. Convexity of a semantic subtyping model and relational parametricity are separate properties.

12.6 Elixir and checked dynamic execution

Elixir’s set-theoretic work uses unions, intersections, refinements, and guarded function behavior. The strong-arrow account pays particular attention to behavior outside the input domain and interaction with dynamic values [17]. It is relevant to CUE’s bridge and legacy-expression boundaries: failure behavior deserves an explicit interpretation alongside the successful result type.

This proposal’s universals are general constraint binders. Its checked effect arrows are specified by chapter 4, rather than identified with Elixir’s strong-arrow construction. Both approaches use set-theoretic descriptions to retain more precise information than an uncorrelated union of inputs and outputs.

12.7 Hyperdoctrines and compact closure

Consider objects that are finite interfaces and morphisms that are definable relations $R \subseteq X \times Y$. Composition is

$$(S \circ R)(x, z) \equiv \exists y. R(x, y) \wedge S(y, z).$$

Juxtaposition of interfaces is the tensor. Equality gives the unit/counit relations making every object self-dual; the snake identities follow by existentially eliminating equality witnesses. This is the compact structure of a relational category.

Adding universal formulas enriches the class of definable relations while retaining these operations, provided the fragment is closed under the required composition and equality relations. Conjunction within one predicate fiber is distinct from the tensor of interfaces. The total executable functions form a separate subcollection and are not asserted to be compact closed.

Lawvere’s adjunctions and the bifibrational account of Melliès and Zeilberger organize these refinement predicates over the relational interface category [7, 8].

Chapter 13

Requirements and implementation outcome

The addendum's section 9 lists six requirements. The last is its restriction on expressive strength, referring to section 7. The first five are evaluated below against the defined reference semantics [6].

13.1 Directionality

A reusable capability is a universal predicate on the implementation. A particular call is a relation on existential argument and result cells. Neither judgment guesses whether a field is provided or consumed. A higher-order parameter introduces its required capability in the caller's type. Module replacement uses its explicitly declared interface and observation relation.

13.2 Monotone refinement

Conjunction refines the joint predicate. Both quantifiers preserve that refinement, by definition 3.1. Dependency-preserving copying and retention of bound variables prevent accidental weakening of universal obligations. An established contradiction remains empty, and an atomic concrete result can only persist or conflict. Chapter 11 implements this property by retaining scoped certificates, residual hypotheses, and validation dependencies. Defaults and comprehension execution retain the separate observation treatment already identified in the reference model.

13.3 Cross-file input and output invariants

Existing argument/result records express existential call restrictions. Universally quantified declarations express reusable behavior and dependent relations. Both can be conjoined across files. chapters 7 and 8 give concrete examples of each form, including length preservation and request-indexed responses.

13.4 Callable self-unification

Function inhabitants have stable semantic origins and captured environments. Their equality is structural on these descriptors, rather than pointer-based. Conjunction is idempotent on their singleton predicates, including closures reached through copying or unrelated record refinement. Partial applications use normalized original-slot bindings. Sealed operation handles preserve the interface's public alias structure.

13.5 Idiomatic foreign interfaces

Portable logical domains remain in public signatures. Representability and conversion failures are explicit checked effects. The existing bridge performs its existing checks and reports those outcomes. Logical kind/domain mismatches are distinguished from in-domain foreign failures. Native widths remain private bridge metadata. No additional conversion stage is required by quantification.

13.6 Expressive strength and implementation milestones

The reference semantics admits higher-rank and dependent specifications. An execution profile may retain the current nonrecursive policy or add certified structural recursion. $\mathcal{U}_1$ is an implementation entry point with explicit generic capabilities and semantic arrow intersections. $\mathcal{S}_1$ adds abstract type witnesses at module boundaries. $\mathcal{S}_H$ extends the same formation rules to arbitrary well-sorted positions.

A conforming implementation establishes the following contracts:

1. Elaboration preserves legacy data observations, lexical binding, copied witness sharing, and quantifier dependencies.
2. The checker establishes the reference judgments soundly and keeps unresolved obligations distinct from contradiction.
3. Symbolic evaluation preserves the successful-completion semantics and the specified observation/effect behavior.
4. The abstraction boundary preserves seal identity, public aliasing, and the equality-respecting simulation used for module evolution.

13.7 Conclusion

Universal and existential binders are dual projections of the same relation-valued constraint semantics. Unification remains intersection. Function types arise by universally constraining an operational application relation; their variance and polymorphic behavior follow from that definition. Dependent contexts and existential modules extend the same structure. The resulting language can be implemented in rank-restricted stages without changing what its quantified descriptions mean.

Appendix A

Core judgments and finite regression model

A.1 Formation and binding summary

A reference type expression has the grammar

$$T, U ::= b \mid a \mid T \wedge U \mid T \vee U \mid \neg T \mid T \times U \mid \text{List}(T) \mid T \Rightarrow U \\ \mid \{\ell_i : T_i\}_{i \in I} \mid \forall a : ST \mid \exists a : ST.$$

Here b is a base predicate; a is a context variable of the appropriate sort; S is a well-formed value sort or type universe, possibly with a guard. Complement is relative to the subject sort. A surface profile may retain only its supported decidable negative predicates. Dependent descriptions extend T_i with references to earlier argument or record cells, using the shared binding graph. Function effects refine the application predicate of chapter 4.

Contexts are telescopes. The free variables of a binder's sort and guard must already occur in its context. A bound variable is identified by its binder, not by its spelling. A quantified expression keeps its subject outside the new binder; an explicit dependent sum keeps the witness as part of the subject.

A.2 Sound introduction and elimination principles

Write $\Gamma; \Delta \vdash e : T$ for a typing judgment in a term environment Γ and a rigid constraint context Δ . The type variable a is fresh for Γ in universal introduction:

$$\frac{\Gamma; \Delta, a : S \vdash e : T}{\Gamma; \Delta \vdash e : \forall a : ST} \quad \frac{\Gamma; \Delta \vdash e : \forall a : ST \quad \Delta \vdash s : S}{\Gamma; \Delta \vdash e : T[s/a]}.$$

The first premise checks the same executable expression at an arbitrary admissible assignment. Bounds appear as assumptions in $\Delta, a : S$.

For transparent existential descriptions, introduction chooses a witness and elimination checks the continuation at an arbitrary opened witness:

$$\frac{\Gamma; \Delta \vdash e : T[s/a] \quad \Delta \vdash s : S}{\Gamma; \Delta \vdash e : \exists a : ST}.$$

Elimination preserves the subject and shares the witness across its projections. Its result may not expose the local witness without an enclosing existential. Opaque packages additionally use the sealing discipline in chapter 9.

Intersection introduction uses the same term:

$$\frac{\Gamma; \Delta \vdash e : T \quad \Gamma; \Delta \vdash e : U}{\Gamma; \Delta \vdash e : T \wedge U}.$$

Subsumption uses semantic inclusion established by a sound derivation. Application requires coverage of the complete supplied-call packet and the stated result or effect obligation. A runtime result check is recorded as an effect unless its success has been established statically.

The proof of the introduction rules is membership in the corresponding intersection or union. Elimination uses the selected instance or a scoped existential witness. These principles specify the reference calculus independently of a particular constraint-solving strategy.

A.3 Regression checks

The following families describe finite reference checks for the algebraic laws and counterexamples. A suitable model enumerates four data values, their sixteen subsets, and 625 partial function tables, with a separate rejection outcome. This checkout does not include the reference-model scripts or their result files; no executed assertion count is claimed here.

Family	Property or counterexample
Arrows	Domain-union and result-intersection identities; contravariance.
Quantified inhabitants	Singleton-rich universal identity; incompatible monomorphic refinement.
Predicate fibers	Both adjunctions; Frobenius reciprocity; inhabited universal floating; quantifier monotonicity.
Scope and bounds	Mixed quantifiers do not commute; empty-range extrusion requires its side condition; subtype bounds retain guarded ranges.
Application	Refinement inclusion holds; meet equality has a counterexample.
Projection	Existential projection need not preserve intersection.
Abstract values	Bijection transport preserves meet, join, equality, refinement, and a finite operation simulation.

Finite checks for the local kernel of [chapter 11](#) should additionally cover preservation of completion sets under bounded propagation, validation of grounded arithmetic cycles, persistence of refutations, retention of guards and existential alternatives, and quantified-function counterexamples. Such reference checks supplement rather than establish correctness of the Go implementation. Executable implementation regressions are indexed in `cue/testdata/quantified/README.md`; supported fragments and checking limits are documented in `doc/implementation.md`. Checker, elaboration, and abstraction obligations are specified in [chapters 10 and 11](#).

Appendix B

Example and feature index

Feature	Location and program forms
Binding syntax	Chapter 5: quantified expressions, declaration parameters, block prefixes, function-local generics, parametric aliases, and instantiation selection.
Pointwise unification	Chapter 3: alpha-renaming, shared subject, constant constraints, guarded bounded clauses, non-distributivity.
Rank-1 programming	Chapter 7: identity, bounded pairing, minimum, map, filter, partition, flat mapping, composition, adapters.
Higher-rank programming	Chapter 7: a polymorphic callback used at two types, returned polymorphic closures, quantified utility records.
Calling conventions	Chapter 7: all parameter modes, defaults, omission, hidden labels, partial application, open protocols, attributes.
Dependent contracts	Chapter 8: exact successor, interval clamp, length-preserving map, safe indexing, append, zip, split, replication.
Dependent structures	Chapter 8: dimension-carrying matrices, transpose, dot product, matrix multiplication, indexed service responses.
Existential reuse	Chapter 9: sealed counters, generic clients, module laws, heterogeneous packages, shared-witness comparison, codecs.
Compatibility	Chapter 12: feature-by-feature correspondence, adapters, old data code, result combination, experimental migration.
Semantic guarantees	Chapters 2 to 4: adjunctions, floating, monotonicity, coverage, variance and application.
Residual checking	Chapter 11: concrete-data validation, scoped certificates, local propagation, witness search, and resource bounds.
Evolution guarantees	Chapter 9: equality-respecting representation simulation, quantified observations, old-compatible views.

References

- [1] The CUE Authors. *CUE language specification*. Living specification, accessed 21 September 2026. <https://cuelang.org/docs/reference/spec/>.
- [2] The CUE Authors. *CUE language specification: References*. Specification of copying and rebinding referenced expressions. Accessed 21 September 2026. <https://cuelang.org/docs/reference/spec/#references>.
- [3] The CUE Authors. *Specifying a default value for a field*. Accessed 21 September 2026. <https://cuelang.org/docs/howto/specify-a-default-value-for-a-field/>.
- [4] The CUE Authors. *Package list*. Standard-library interface documentation, accessed 21 September 2026. <https://pkg.go.dev/cuelang.org/go/pkg/list>.
- [5] Marcel van Lohuizen. *Proposal: functions in CUE*. 10 July 2026. CUE proposal 4484, draft at repository revision `a949162aedb86661183c46abef47919d16605014`. <https://github.com/cue-lang/proposal/blob/a949162aedb86661183c46abef47919d16605014/designs/language/4484-functions.md>.
- [6] Marcel van Lohuizen. *Why CUE Functions Tighten Instead of Widen*. Theory addendum to proposal 4484, committed 19 September 2026, repository revision `a949162aedb86661183c46abef47919d16605014`. <https://github.com/cue-lang/proposal/blob/a949162aedb86661183c46abef47919d16605014/designs/language/4484-theory-addendum.md>.
- [7] F. William Lawvere. Adjointness in foundations. *Dialectica*, 23(3–4):281–296, 1969. Reprinted with commentary in *Reprints in Theory and Applications of Categories*, 16:1–16, 2006. <https://tac.mta.ca/tac/reprints/articles/16/tr16abs.html>.
- [8] Paul-André Melliès and Noam Zeilberger. *A bifibrational reconstruction of Lawvere’s presheaf hyperdoctrine*. 2016. arXiv:1601.06098. <https://arxiv.org/abs/1601.06098>.
- [9] John C. Reynolds. Polymorphism is not set-theoretic. In *Semantics of Data Types*, LNCS 173, pages 145–156, 1984. https://doi.org/10.1007/3-540-13346-1_7.
- [10] John C. Reynolds. Types, abstraction and parametric polymorphism. In *Information Processing 83*, pages 513–523. North-Holland, 1983. <https://www.cs.cmu.edu/~crary/819-f09/Reynolds83.pdf>.
- [11] Alain Frisch, Giuseppe Castagna, and Véronique Benzaken. Semantic subtyping: Dealing set-theoretically with function, union, intersection, and negation types. *Journal of the ACM*, 55(4), Article 19, 2008. <https://doi.org/10.1145/1391289.1391293>.
- [12] Giuseppe Castagna and Zhiwu Xu. Set-theoretic foundation of parametric polymorphism and subtyping. In *ICFP*, pages 94–106, 2011. <https://doi.org/10.1145/2034773.2034788>.
- [13] Giuseppe Castagna, Kim Nguyen, Zhiwu Xu, Hyeonseung Im, Sergueï Lenglet, and Luca Padovani. Polymorphic functions with set-theoretic types. Part 1: Syntax, semantics, and evaluation. In *POPL*, pages 5–18, 2014. <https://doi.org/10.1145/2535838.2535840>.
- [14] Giuseppe Castagna, Kim Nguyen, Zhiwu Xu, and Pietro Abate. Polymorphic functions with set-theoretic types. Part 2: Local type inference and type reconstruction. In *POPL*, pages 289–302, 2015. <https://doi.org/10.1145/2676726.2676991>.

- [15] The CDuce Project. *Polymorphism tutorial*. Accessed 21 September 2026. https://www.cduce.org/tutorial_polymorphism.html.
- [16] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyen. Polymorphic type inference for dynamic languages. *Proceedings of the ACM on Programming Languages*, 8(POPL), 2024. <https://doi.org/10.1145/3632882>. Preprint: <https://arxiv.org/abs/2311.10426>.
- [17] José Valim. *Strong arrows: a new approach to gradual typing*. 20 September 2023. <https://elixir-lang.org/blog/2023/09/20/strong-arrows-gradual-typing/>.
- [18] John C. Mitchell and Gordon D. Plotkin. Abstract types have existential type. *ACM Transactions on Programming Languages and Systems*, 10(3):470–502, 1988. <https://doi.org/10.1145/44501.45065>.
- [19] John C. Mitchell. Representation independence and data abstraction. In *POPL*, pages 263–276, 1986. <https://doi.org/10.1145/512644.512669>.
- [20] Jana Dunfield and Neelakantan R. Krishnaswami. Complete and easy bidirectional typechecking for higher-rank polymorphism. In *ICFP*, pages 429–442, 2013. <https://doi.org/10.1145/2500365.2500582>. Preprint: <https://arxiv.org/abs/1306.6032>.
- [21] Gopalan Nadathur, Bharat Jayaraman, and Keehang Kwon. *Scoping constructs in logic programming: Implementation problems and their solution*. 1998. arXiv:cs/9809016. <https://arxiv.org/abs/cs/9809016>.
- [22] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid types. In *PLDI*, 2008. https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.
- [23] The GHC Developers. *GHC User’s Guide: Arbitrary-rank polymorphism*. Accessed 21 September 2026. https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/rank_polymorphism.html.
- [24] The CUE Authors. *Reference Cycles*. CUE language tour, accessed 22 September 2026. <https://cuelang.org/docs/tour/references/cycle/>.
- [25] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pages 238–252, 1977. <https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml>.
- [26] Vijay A. Saraswat, Martin Rinard, and Prakash Panangaden. The semantic foundations of concurrent constraint programming. In *POPL*, pages 333–352, 1991. <https://doi.org/10.1145/99583.99627>.